



Original article

DOAL-IDS: Deep Optimized Active Learning Framework for Intrusion Detection in IoT Systems

Aymin Javed ^a, Nadeem Javaid ^a,^{*}, Imran Ahmed ^b, Badr Alsamani ^c,^{*},
Abdul Khader Jilani Saudagar ^c

^a ComSens Lab, International Graduate School of Artificial Intelligence, National Yunlin University of Science and Technology, Douliu, Yunlin 64002, Taiwan

^b School of Computing and Information Science, Anglia Ruskin University, Cambridge CB11PT, UK

^c Information Systems Department, College of Computer and Information Sciences, Imam Mohammad Ibn Saud Islamic University (IMSIU), Riyadh 11432, Saudi Arabia

ARTICLE INFO

Dataset link: <https://www.kaggle.com/datasets/fadiabuzwayed/ton-iot-train-test-network>

Keywords:

Internet of Things
Intrusion Detection
ToN_IoT
Capsule Network
Reptile search optimization
Marginal-based active learning
eXplainable Artificial Intelligence

ABSTRACT

Internet of Things (IoT) environments are difficult to detect intrusion because of the heterogeneity of the devices and intricate attack patterns. Class imbalance, irrelevant features, suboptimal accuracy, poor hyperparameter tuning, and limited labeling data are some of the problems that exist in traditional machine and deep learning models. To address these challenges, we propose the Deep Optimized Active Learning Framework for Intrusion Detection in IoT Systems (DOAL-IDS). First, low-variance features are removed using a variance threshold, and the proximity weighted random affine shadow sampling technique balances the dataset by generating synthetic samples near the minority class. Capsule Network (CapsNet) is employed for classification and further optimized using the Reptile Search Algorithm (RSA), forming Reptile-Optimized Capsule Network (ROC-Net). ROC-Net is enhanced with Margin-Based Active Learning (MBAL) to create MARCO-Net, which efficiently annotates the most uncertain samples, reducing labeling costs. Experimental results show that ROC-Net and MARCO-Net achieve improvements of 8.75% and 12.5% in accuracy, 7.41% and 11.11% in F1-score, and 11.94% and 20.90% in Matthews correlation coefficient, with reductions of 75.86% and 82.70% in log loss. Robustness is ensured via 10-fold cross-validation and the Wilcoxon signed-rank test. For interpretability, local interpretable model-agnostic explanations and Shapley additive explanations analyzes provide insights into model decisions. These results demonstrate the effectiveness, reliability, and transparency of the proposed DOAL-IDS framework for IoT intrusion detection.

1. Background

1.1. Introduction

Internet of Things (IoT) has become a revolutionary paradigm which has facilitated smooth interconnectivity of heterogeneous devices including sensors, actuators, smart appliances and embedded systems [1]. These gadgets work together to gather, send and process information to support intelligent decision-making in various industries, such as in healthcare, smart homes, automation in industries, or transport among others [2,3]. The boom in the IoT devices has significantly increased the digital ecosystem, which is incredibly convenient, real-time, and automated [4].

IoT networks are vulnerable to significant security risks because their decentralized and resource-constrained architecture implies substantial security challenges. The IoT environment does not tend to support traditional security solutions due to the limited compute, memory,

and energy resources. This variant of devices and open communication protocols also increases the attack surface and leads to the high vulnerability of IoT systems to a number of cyber threats, such as malware injection, spoofing, Denial-of-Service (DoS) attacks [5,6], and man-in-the-middle attacks.

In addition to these common threats, there are also risks of advanced persistent threats, blackhole, grayhole, botnet-based large-scale Distributed Denial-of-Service (DDoS) campaigns, side-channel attacks that utilize weaknesses at the hardware level, among others in IoT ecosystems [7]. Devices are frequently compromised by attackers using weak authentication, old-fashioned firmware, or poorly designed APIs, and used to access larger networks. These multi-vector attacks increase the challenge of providing security, not only due to the heterogeneity of IoT devices that increases the attack face, but also due to the increased complexity in detection and mitigation planning.

^{*} Corresponding author.

E-mail addresses: javidn@yuntech.edu.tw (N. Javaid), bsalsamani@imamu.edu.sa (B. Alsamani).

<https://doi.org/10.1016/j.aej.2026.02.013>

Received 16 September 2025; Received in revised form 27 January 2026; Accepted 11 February 2026

Available online 4 March 2026

1110-0168/© 2026 The Authors. Published by Elsevier B.V. on behalf of Faculty of Engineering, Alexandria University. This is an open access article under the CC BY-NC-ND license (<http://creativecommons.org/licenses/by-nc-nd/4.0/>).

Table 1
List of acronyms and description.

Acronyms	Description
10-FCV	10-Fold Cross Validation
CapsNet	Capsule Network
DL	Deep Learning
IDS	Intrusion Detection System
IoT	Internet of Things
LIME	Local Interpretable Model-agnostic Explanations
MBAL	Margin-Based Active Learning
MCC	Matthews Correlation Coefficient
ProWRAS	Proximity Weighted Random Affine Shadow Sampling
ProWRAS	Proximity Weighted Random Affine Shadow Sampling
RSA	Reptile Search Algorithm
SHAP	SHapley Additive exPlanations
VT	Variance Threshold
x	Original feature value
x'	Normalized feature value after Min–Max Scaling
n	Total number of samples
μ_j	Mean of the j th feature
$Var(X_j)$	Variance of the j th feature
C_k	Cluster k of minority class instances
$ S $	Number of selected shadow samples
x_{syn}	Synthetic sample generated from shadow samples
μ_M	Mean of majority class instances
$\mathbf{v}_j$	Output vector of capsule j after squashing
$\mathbf{s}_j$	Total input to capsule j
c_{ij}	Routing coefficient from capsule i to capsule j
$\mathbf{u}_i$	Output vector of capsule i
$\mathbf{W}_{ij}$	Weight matrix between capsule i and capsule j
$\mathbf{X}_i^t$	Position (solution vector) of i th candidate at iteration t
$\mathbf{X}_{best}^t$	Best solution found at iteration t

IoT-specific robust Intrusion Detection Systems (IDS) are thus important in addressing such challenges [8]. However, traditional IDS have difficulties in functioning within the limits of IoT systems. Low sensitivity of a majority of the modern systems to hidden threats makes it worse and thus results in high false positive rates and delays in detecting a threat.

The recent studies have given more attention to the use of Machine Learning (ML) and Deep Learning (DL) to improve IDS in IoT networks. These methods acquire intricate patterns using high-dimensional traffic data and evolve with the dynamically changing cyber threat. In contrast to the traditional rule-based schemes, ML and DL models can identify known as well as zero-day attacks by identifying minor abnormalities in network behavior. Furthermore, advanced architectures such as Convolutional Neural Networks (CNNs) [9], Recurrent Neural Networks (RNNs) [10], and hybrid models have shown strong potential to improve detection accuracy, minimizing False Positives (FP), and ensure scalability across heterogeneous IoT environments. Consequently, the incorporation of smart learning-oriented practices has emerged as a fundamental benchmark in the creation of the next-generation and robust IDS adapted to the IoT ecosystems. List of abbreviations and symbols is given in Table 1.

1.2. Related work

The rapid growth of IoT networks has motivated researchers to design IDS that can cope with diverse attack patterns and evolving security threats. A wide range of studies has explored both traditional and modern approaches to enhance detection accuracy and resilience in IoT environments.

The authors in [11] proposed an innovative model for IDS that employs a two-layer dimensionality reduction and a two-tier classification module, aimed at identifying malicious actions, including user to root and remote to local attacks. Component analysis and linear discriminant analysis of the dimension reduction module are used in the proposed model to reduce the high-dimensional dataset to a lower one with fewer features. The authors then apply a two-level

classification module that utilizes Naive Bayes (NB) and the certainty factor version of K-Nearest Neighbor (KNN) in identifying anomalous behavior. Strength: The suggested model is a good merger between dimensionality reduction and dual-level classification approach that improves the level of detection by utilizing the NB and the certainty factor version of KNN. Limitations: The model has several limitations, even though it has a better detection ability, it uses the classic classifiers that might not be scalable in a very large data set and might not adapt with the changing patterns of attacks in the contemporary intrusion detection context.

In [12], the authors introduced a novel anomaly-based IoT network IDS that used DL methods. It has proposed a filter-based feature selection Deep neural network (DNN) model, which removes highly correlated features. The model is also optimized in terms of different parameters and hyperparameters tuning. Evaluation is performed on the UNSW-NB15 dataset that has four types of attacks. Strength: The suggested IDS uses a filter-based feature selection system with DNN, which minimizes the redundant features by excluding very correlated features and increases the efficiency of the model. Limitation: Although the model is optimized through parameter and hyperparameter tuning, its evaluation is limited to the UNSW-NB15 dataset, which may restrict its generalizability to other IoT attack scenarios.

In [13], the authors present a new hybrid IDS, which is based on CNN and Long Short-Term Memory (LSTM) architectures and the Reptile Search Algorithm (RSA). The proposed method uses the CNN component to extract spatial features and LSTM component to predict temporal dependencies in IoT traffic to achieve thorough detection of anomalies. The experimental findings indicate that RSA-enhanced IDS is superior to the conventional optimization algorithms like particle swarm optimization since it has a faster convergence speed and better performance metrics.

In [14], the authors proposed an IDS for IoT networks that categorizes traffic flow using DL models. IoT dataset is utilized and general features are from the field information at the packet level. A feed-forward neural network model is constructed for binary and multi-class classification, encompassing DoS, DDoS, reconnaissance, and information theft attacks targeting IoT devices. Strength: The proposed IDS demonstrates flexibility by performing both binary and multi-class classification, effectively detecting a wide range of IoT-specific attacks such as DoS, DDoS, reconnaissance, and information theft. Limitation: The reliance on packet-level general features may overlook more complex traffic patterns, potentially limiting the model's capability to detect sophisticated or evolving attack types.

The study [15] suggests a new method of improving the Quality of Service (QoS) parameters in IoT with the help of ML and blockchain technologies. An NS2 simulator is used to simulate the IoT network with the help of blockchain technology. Various QoS parameters including delay, throughput, packet delivery ratio and packet loss are examined. Various ML models including NB, Decision Tree (DT), and ensemble, learning methods are used to classify the obtained QoS values. Results show that the ensemble classifier has the best classification accuracy of 83.74% as compared to the NB and DT classifiers. Strength: The integration of ML with blockchain technology provides a novel approach to enhancing QoS in IoT networks, with ensemble learning demonstrating superior classification accuracy over traditional models. Limitation: The evaluation is limited to simulation-based experiments using NS2, which may not fully capture the complexities and scalability challenges of real-world IoT environments.

The authors in [16] presents an optimized ML-based IDS in the field of IoT. It is based on a set of high quality real-time data to train and test the performance of the proposed ML-based model and compare its results with a subset of ML algorithms. The study evaluated several optimizable algorithms, optimizable tree, optimizable discriminant, optimizable KNN, Logistic Regression (LR) kernel and optimizable Ensemble. The proposed model is tested on the NLS-KDD dataset using the MATLAB platform, and its effectiveness is assessed

across various performance metrics. Strength: The proposed IDS leverages high-quality real-time data along with optimizable ML algorithms, which enhances the robustness and reliability of the evaluation across multiple performance metrics. Limitation: The reliance on the NSL-KDD dataset and MATLAB-based experimentation may restrict the generalizability and scalability of the results to diverse and large-scale IoT environments.

In [17], the authors present a hybrid ML solution that applies KNN and Random Forests (RF) as supervised classifiers to improve the accuracy of IDS and reduce the threat of possible attacks. In addition, they use backward elimination and the Linear Discriminant Analysis (LDA) algorithms to reduce the features and to reduce the cost of computation. After the training period, when there are differences between the decisions of the classifiers, the final decision is supported with ISO/IEC 27001 regulations. The proposed model is evaluated in a Python environment with the CICIDS 2017, NSL-KDD, and TON-IoT datasets. Strength: The hybrid approach combining KNN and RF, along with feature reduction using backward elimination and LDA, enhances the detection accuracy while reducing computational overhead. Limitation: The reliance on ISO/IEC 27001 regulations for resolving classifier disagreements may limit the model's adaptability to diverse IDS scenarios that fall outside the standard's scope.

In [18], the authors examine the security issues of IoT, existing IDS, and the use of DL to detect intrusion in IoT networks. The BoT-IoT dataset is trained on 21 neural network models and an ensemble learning model. Performance is measured in terms of the standard classification metrics. The proposed model has a higher accuracy, recall, and F1-score than the previous studies by 0.005, 0.55 and 0.84, respectively. These results show that the security of the IoT networks can be enhanced successfully by employing the DL-based methods. Strength: The 21 neural network models and an ensemble model are developed and evaluated, which comprehensively explore the use of DL-based methods in intrusion detection in the IoT and obtain significant improvements in accuracy, recall, and F1-score. Limitations: Multiple-deep training models can be computationally costly and might need significant hardware investment, which may not be feasible in fine-resource IoT applications.

In [19], the authors introduced a novel hybrid ML framework that integrates KMeans-Synthetic Minority Over-sampling Technique (SMOTE) (KMS) to balance the data and Principal Component Analysis (PCA) to reduce the dimensions, wherein the framework is evaluated in WSN-DS and TON-IoT datasets. The models employed in the model are DT, RF and gradient boosting models such as eXtreme Gradient Boosting (XGB) to improve the accuracy and efficiency of detection. Strength: Data balancing using KMeans-SMOTE with dimensionality reduction using PCA improves the detection performance of a variety of classifiers and the computation efficiency of the algorithm. Limitations: The use of synthetic data generation via SMOTE could add noise or bias to the training process and this could compromise the applicability of the model in the real world.

In [20], the authors normalize preprocessing of IEEE Dataport input data and employ SMOTE to deal with the imbalance of classes. Subsequently, the Vortex Search Optimization Algorithm (VSA) is used to improve the accuracy of classification by selecting the various features in the best way possible. Finally, the input samples are classified using a Fine-tuned based Support Vector machine (FT-SVM). To find out the performance of the given model, it is compared against a common set of data. The effectiveness of the proposed model on the chosen dataset is proven by the results of the experiment. Strength: VSA area of feature selection with a FT-SVM is an effective way to enhance the accuracy of the classification and it shows high performance with the presented dataset. Limitations: The analysis is conducted on one dataset, and this might not be sufficient to test the strength of the model to different intrusion patterns or different IoT contexts.

On the whole, the studies reviewed in this paper indicate the high importance of smart IDSs in the protection of IoT ecosystems.

Although significant progress has been achieved, the current literature still has issues with interpretability, adaptability, and practical implementation, which opens numerous opportunities of advancement and development. A summary of the related work is provided in Table 2.

According to the literature reviewed, a number of research gaps exist that are critical and encourage the contributions of the study. First, most of the available IDS have redundant and irrelevant features, which results in high dimensionality and low model efficiency. The imbalance in classes is another significant issue, in which the lack of attack types underrepresentation leads to biased learning and inability to detect the rare but significant intrusions. In addition, earlier models of IDS have an insufficient ability to identify complex hierarchical relations among network traffic, which restricts the overall performance in terms of classification. In addition, most studies rely on manual hyperparameter tuning, which is computationally expensive and frequently yields suboptimal configurations that fail to generalize across diverse IoT traffic scenarios. The expensive nature and lack of labeled data are also often under consideration, as most methods assume that an entire dataset of annotations is available, which is hardly realistic in an actual IoT system. In addition, research conducted in the past has inadequate validation practices in which there is little or no application of criteria that are strong evaluation methods or statistical tests to determine the reliability of results. Lastly, transparency and interpretability are not highly regarded and many IDS solutions remain black-box models that are not very informative about how they make decisions.

1.3. Major contributions

The main findings of this research are as follows. First, Variance Threshold (VT) is employed for feature selection method to eliminate redundant features and keep only the most informative features reducing overall data dimensionality. To solve the problem of imbalance between classes, Proximity Weighted Random Affine Shadow Sampling (ProWRAS) is used to produce boundary aware synthetic samples that maintain the underlying decision boundary. For effective feature extraction and classification, a Capsule Network (CapsNet) is utilized due to its strong capability to capture hierarchical relationships within network traffic data. Moreover, the Reptile Search Algorithm (RSA) is applied in the hyperparameter optimization, which leads to the creation of the Reptile-Optimized Capsule Network (ROC-Net). Expanding on this, a new model is proposed called Marginal Active-learning with Reptile-Optimized Capsule Network (MARGO-Net), that combines Margin-Based Active Learning (MBAL) to iteratively sample and label the most uncertain samples so that in addition to cutting down on the cost of labeling, it makes learning more efficient. To evaluate the two proposed models effectively, the 10-Fold Cross-Validation (10-FCV) is used to validate the models, and the Wilcoxon Signed-Rank Test is used to determine statistical significance. Lastly, SHapley Additive exPlanations (SHAP) and Local Interpretable Model-agnostic Explanations (LIME) are used to facilitate transparency and interpretability and offer significant insights into how the proposed models make decisions. Overall, these contributions collectively advance the development of a more efficient, reliable, and interpretable intrusion detection framework for IoT environments.

This study extends our previous work [21] by adding new comparisons, statistical analyses, confidence intervals, and explainable AI techniques such as SHAP and LIME, covering 70% new content.

The rest of the paper is organized as follows: Section 2 presents the proposed DOAL-IDS framework. Section 3 discusses the results and provides a detailed analysis. Finally, Section 4 concludes the paper.

2. Proposed DOAL-IDS framework

The proposed Deep Optimized Active Learning Framework for IDS in IoT Systems (DOAL-IDS) framework begins with data preprocessing, where categorical variables are transformed using label encoding and

Table 2
Summary of related work.

Methodology	Limitations	Our solution
Anomaly-based IDS using DNN with filter-based feature selection; correlated features removed and model optimized on UNSW-NB15 dataset [12].	Focused only on correlation removal; redundant and low-variance features still remain, reducing efficiency. Limited dataset diversity.	VT applied to retain only informative features, reducing redundancy and dimensionality.
Feed-forward Neural Network for binary and multi-class IDS using IoT dataset with packet-level feature extraction [14].	Limited feature engineering and generalization. Struggles to capture hierarchical feature relationships in traffic data.	CapsNet captures hierarchical and spatial relationships, improving representation power.
QoS-aware IDS using ML (Naïve Bayes, DT, Ensemble) integrated with Blockchain; simulated on NS2 [15].	Simulation-based, lacks real-world validation. Focus only on QoS metrics, ignoring class imbalance and complex attack patterns.	ProWRAS applied for realistic data balancing, preserving decision boundaries. Evaluation performed on real ToN-IoT dataset.
Optimized ML-based IDS with tree, KNN, LR, and ensemble classifiers on NSL-KDD dataset [16].	Dependent on traditional ML models; limited scalability for high-dimensional IoT data. Hyperparameters tuned manually.	Hyperparameter optimization via RSA improves convergence and generalization in CapsNet.
Hybrid ML solution using KNN and RF with backward elimination and LDA feature reduction, validated on CICIDS2017, NSL-KDD, and TON-IoT [17].	High computational cost of multiple classifiers. Limited adaptability to new attack patterns.	ROC-Net provides efficient hyperparameter-optimized classification with better generalization.
DL-based IDS trained on Bot-IoT dataset with 21 models and ensemble learning [18].	High training cost, sensitive to hyperparameters, limited interpretability.	RSA-tuned ROC-Net ensures stability; SHAP and LIME enhance transparency and interpretability.
Hybrid ML model combining KMeans-SMOTE for balancing and PCA for dimensionality reduction with DT, RF, and XGB classifiers on WSN-DS and TON-IoT [19].	SMOTE introduces noisy synthetic data near decision boundaries; PCA often discard useful information.	ProWRAS generates boundary-aware synthetic samples, preserving class separability.
SMOTE with IEEE Dataport dataset; features optimized using VSA and classified via FT-SVM [20].	Relies on SMOTE, prone to overfitting and poor handling of boundary samples. Limited scalability beyond SVM.	ProWRAS replaces SMOTE with boundary-preserving sampling. ROC-Net and MARCO-Net improve scalability and efficiency.

numerical attributes are normalized through min–max scaling to ensure uniformity across features. A VT is then applied to retain only the most informative features, reducing redundancy and dimensionality. After that, to resolve class imbalance, the ProWRAS oversampling technique is employed, generating boundary-aware synthetic samples. For classification, CapsNet is utilized due to its ability to capture hierarchical feature representations. Furthermore, an ROC-Net is proposed to improve predictive performance. In addition, MARCO-Net is employed for efficient data labeling using MBAL, where the most uncertain samples are iteratively annotated to strengthen the CapsNet model with minimal labeling effort. The complete workflow of the proposed framework is illustrated in Fig. 1. The methodology used in this study is summarized in Table 3.

2.1. ToN-IoT dataset description

The ToN IoT dataset, presented by IoT Lab at UNSW Canberra Cyber, is designed to assess cybersecurity model in IIoT contexts [22]. Based on a realistic testbed, it incorporates a number of data sources, such as the network traffic and telemetry, to mirror real-life attack cases. This whole dataset facilitates both supervised and unsupervised learning with structured and labeled data. It includes 45 features within a wide range of normal and attack behavior. In order to provide efficiency in the processing and maintain data diversity, the study used a subset of 50,000 records to train and evaluate the methods of ML and DL IDS.

2.2. Data preprocessing and sampling for intrusion detection

Network intrusion detection data is loaded through Google Drive. Preliminary examination showed a number of columns, such as source and destination IP addresses, ports, timestamps, and the label to target. Columns such as *src_ip*, *src_port*, *dst_ip*, *dst_port*, and *ts* are dropped, as they do not contribute directly to classification and may introduce noise

or redundancy. The feature matrix X is defined by excluding the target column, while the target vector y is set as the *type* column, representing the category of network traffic. Stratified sampling is conducted in order to downsample the dataset with preserving the original distribution of classes. This makes sure that both majority and minority classes are proportionate in the sampled subset which is particularly significant when dealing with imbalanced datasets. To simplify the problem, the multi-class problem is reduced to a binary classification task. All attack types are merged into a single class *attack*, while normal traffic is kept as *normal*. This transformation aids in specializing in the detection of malicious activity as opposed to benign activity. The updated final feature matrix and target vectors are set up to be used in the further preprocessing, scaling, and modeling processes and are prepared by the binary classification arrangement.

2.2.1. Label encoding

Categorical features cannot be directly interpreted by DL models, as most algorithms require numerical input [23]. Therefore, categorical features are transformed into numerical form using label encoding, where each distinct category is mapped to a unique integer, as shown in block (2) in Fig. 2. This process can be expressed using Eq. (1):

$$LE(c_i) = v_j, \quad \text{where } c_i \in C, v_j \in \{0, 1, \dots, |C| - 1\}, \quad (1)$$

where C denotes the set of unique categories, c_i is the i th category, and v_j is the corresponding integer label. This transformation enables ML and DL models to process categorical variables efficiently without introducing artificial order.

2.2.2. Min–max scaling

Since numerical features often have different ranges and scales, models often become biased toward attributes with larger magnitudes. To address this issue, numerical features are normalized using min–max scaling, which linearly transforms data into a fixed range $[0, 1]$, as

Table 3
Methodology utilized in the study.

Method	Description	Key features
Label encoding	Transforms categorical variables into numerical form by assigning each unique category a distinct integer.	Simple transformation, preserves category uniqueness, avoids artificial ordering.
Min–Max scaling	Normalizes numerical features into a fixed range, typically [0,1], ensuring uniform magnitude across variables.	Preserves relative distances, accelerates convergence, sensitive to outliers.
VT	Removes low-variance features that carry little discriminatory information, keeping only statistically meaningful attributes.	Reduces dimensionality, eliminates redundant/noisy features, improves model efficiency.
ProWRAS	Generates boundary-aware synthetic minority samples using proximity-weighted affine shadow sampling to handle class imbalance.	Preserves decision boundaries, avoids noisy oversampling, enhances minority-class representation.
CapsNet	Advanced neural network architecture that represents features as vectors rather than scalars, capturing hierarchical relationships.	Routing-by-agreement, robust to pose/scale/rotation changes, better feature representation than CNNs.
RSA	Metaheuristic inspired by crocodile hunting strategies for hyperparameter optimization of CapsNet.	Balances exploration and exploitation, avoids local optima, improves convergence and generalization.
MBAL	Iteratively selects the most uncertain samples (smallest probability margin between top predictions) for labeling.	Reduces labeling cost, focuses on decision boundary, improves generalization with fewer samples.
10-FCV	Splits dataset into 10 equal folds, using each once as a test set while training on the remaining folds.	Reduces bias/variance, robust performance estimate, widely accepted validation strategy.
SHAP	Explains predictions by distributing contribution of each feature fairly using Shapley values from cooperative game theory.	Global and local interpretability, consistent and theoretically grounded explanations.
LIME	Provides local explanations by approximating the model with an interpretable surrogate around the instance of interest.	Model-agnostic, instance-level interpretability, highlights feature importance locally.

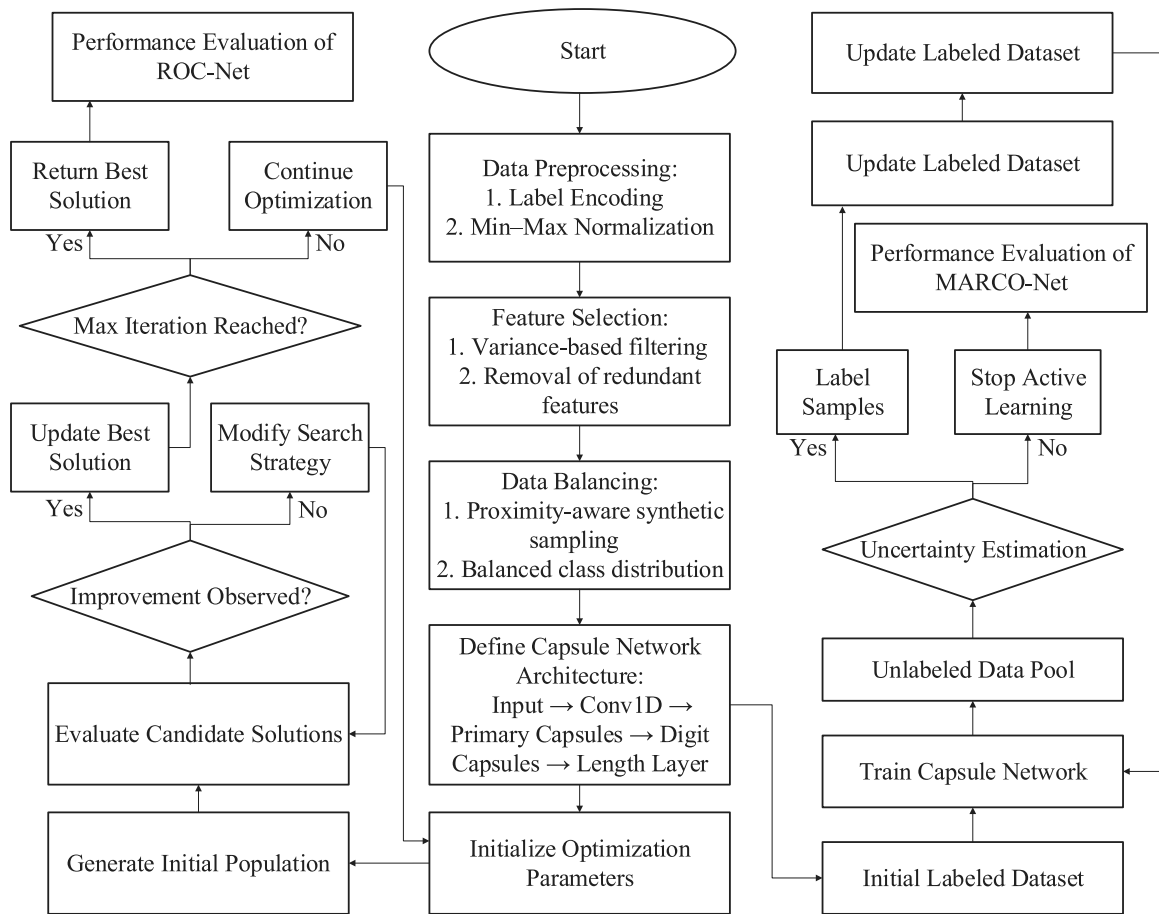


Fig. 1. Flow chart of proposed Deep Optimized Active Learning Framework for intrusion detection in IoT systems.

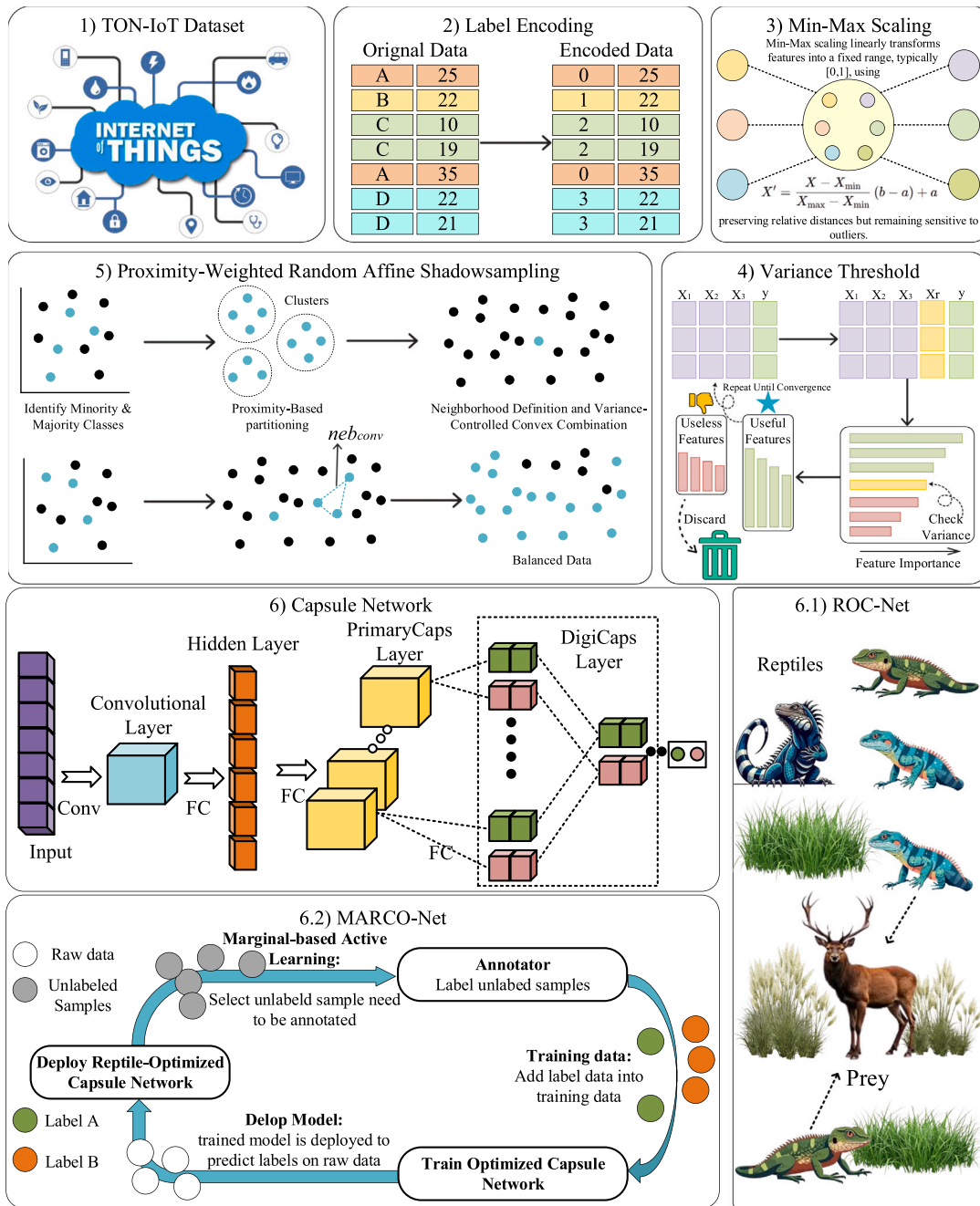


Fig. 2. Proposed Deep Optimized Active Learning Framework for intrusion detection in IoT systems.

illustrated in block (3) in Fig. 2. The transformation is defined in Eq. (2):

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}, \quad (2)$$

where x is the original feature value, and $x_{\min}$ and $x_{\max}$ are the minimum and maximum values of the feature, respectively. This normalization ensures uniform feature values, improves gradient-based optimization, and accelerates convergence [24].

2.3. Feature selection using variance threshold

VT is a simple and effective feature selection technique used to remove features with low variance, as such features are unlikely to carry meaningful information for classification tasks [25]. By setting a threshold (e.g., 0.01), any feature whose variance does not exceed

this threshold across the dataset is eliminated, as shown in block (4) in Fig. 2. The variance of a feature X_j is computed using Eq. (3):

$$Var(X_j) = \frac{1}{n} \sum_{i=1}^n (x_{ij} - \mu_j)^2, \quad (3)$$

where x_{ij} is the value of the j th feature for the i th sample, μ_j is the mean of the j th feature, and n is the total number of samples. A feature X_j is retained for further processing if $Var(X_j) \geq \tau$, where τ is the predefined threshold. This helps reduce noise and dimensionality, allowing the model to focus on more informative features. In this study, VT is applied after encoding and scaling the features of the ToN_IoT dataset. As a result, a subset of features with sufficient variability is retained for further analysis. The selected features include: *proto*, *service*, *duration*, *src_bytes*, *dst_bytes*, *conn_state*, *src_ip_bytes*, *dst_ip_bytes*, *dns_query*, *dns_qclass*, *dns_qtype*, *dns_rcode*, *dns_AA*, *dns_RD*,

dns_RA, *dns_rejected*, *label*, and *type*. The selected features are considered the most informative and relevant for IDS in the IoT environment, improving the model's ability to learn meaningful patterns from the data.

2.4. Data balancing with Proximity Weighted Random Affine Shadow Sampling (ProWRAS)

Algorithm 1 ProWRAS Resampling Procedure

Require: Feature set X , label set y

Ensure: Balanced feature set X_{res} , balanced labels y_{res}

- 1: Initialize ProWRAS object: $\text{lrs} \leftarrow \text{ProWRAS}()$
- 2: Identify minority and majority classes in y
- 3: Compute proximity weights for minority samples
- 4: Generate affine shadow points using the weighted sampling strategy
- 5: Create synthetic samples by combining shadow points with neighborhood information
- 6: Combine original samples with generated synthetic samples
- 7: Return the resampled dataset: $(X_{\text{res}}, y_{\text{res}})$

ProWRAS operates by identifying local clusters of minority class instances that lie close to the decision boundary, where the risk of misclassification is highest. It assigns greater importance to clusters near majority class samples using a spatial proximity-based weighting mechanism [26]. From each cluster C_k , a subset $S \subset C_k$ of shadow samples is selected, guided by the parameter *max-conv*, as shown in block (5) of Fig. 2. A synthetic sample is then generated using Eq. (4):

$$x_{\text{syn}} = \frac{1}{|S|} \sum_{x \in S} x, \quad (4)$$

This equation computes the synthetic sample x_{syn} as the average of all selected shadow samples in the set S . In other words, it represents the centroid of the shadow samples, capturing their collective characteristics. To preserve class separability, the variation among synthetic samples is controlled using the constraint, as Eq. (5) shows:

$$\|x_{\text{syn}} - \mu_M\| > \delta, \quad (5)$$

where μ_M is the mean of the majority class instances and δ is a distance threshold that enforces separation from the majority class. This controlled generation mechanism ensures that the synthetic minority samples lie close to the decision boundary without overlapping the majority class, thereby improving class balance while avoiding noisy or redundant instances. The working of ProWRAS is given in Algorithm 1. We use ProWRAS with the default settings given by the *pyloras* library to implement it. In particular, shadow points number and affine transformation factor are maintained at their default values, proximity weighting is activated, the neighborhood size (k) is used as the default value, and the random state parameter is not adjusted by hand. These environments allowed the reproducibility and consistency of resampling behavior across the experiments. In IoT environments, where attack samples are often scarce, ProWRAS significantly enhances the robustness of IDS models in real-world imbalanced scenarios. In comparison to conventional oversampling methods like SMOTE, which blend randomly-selected minority pairs together linearly, ProWRAS can utilize proximity at the cluster level to index the most informative parts of the feature space. This focused augmentation enhances the discriminative ability of classifiers because it strengthens the boundaries of the discriminated decisions instead of spreading them into spurious synthetic points. Moreover, the weighting process prevents the risk of overfitting to synthetic noise, which is a weakness of random oversampling strategies.

Table 4

Capsule network basic architecture.

Layer type	Details
Input layer	(features, 1)
Conv1D (Conv1)	125 filters, kernel size = 3, activation = ReLU
Conv1D (Primary capsule)	256 filters (8×32), kernel size = 9, strides = 2
Reshape (Primary capsule)	Reshape to $(-1, 8)$
Squash activation (Primary capsule)	Activation = Squash
Capsule layer	2 capsules, 16 dimensions, routings = 1
Optimizer	RMSprop
Loss function	Binary crossentropy

A second advantage of ProWRAS is that it can be used with high-dimensional data, where the probability of overlapping classes is increased. With the distance threshold, δ , the approach provides a protective measure against the production of samples that are too close to majority class areas, thereby maintaining inter-class distances. This property causes ProWRAS to be well adapted to the IDS application where minor behavior variations must be learned without the classifier being flooded with spurious minority examples. In general, ProWRAS balance the classes distributions as well as enhance the geometric quality of the synthetic data that results in better generalization, fewer FPs, and the ability to detect more of the rare attack patterns.

2.5. Proposed Reptile-Optimized Capsule Network (ROC-Net)

CapsNet is an advanced neural network architecture introduced to overcome the limitations of traditional CNNs. Unlike CNNs, which produce scalar activations, CapsNet produces vectors, the length of which quantifies the likelihood of feature occurrence and the orientation quantifies spatial characteristics such as pose, rotation, and scale. This enables CapsNet to maintain hierarchical relationships between features, and thus is especially effective when applied to complex data domains like IoT intrusion detection.

Primary capsules (combination of low-level feature maps into vectors) are followed by a convolutional layer as the first stage of CapsNet. The different properties of patterns that are being detected are coded in each primary capsule; some of these properties include orientation and position, thus making them the basic blocks of higher level feature representation. These vectors are then passed to the digit capsules (or class capsules), which aggregate information from primary capsules through a dynamic routing mechanism, as given in Algorithm 2. Each digit capsule corresponds to a class label and generates a vector whose length indicates the probability of that class being present in the input. The orientation of the vector preserves instantiation parameters, allowing CapsNet to distinguish between objects even under rotation, scaling, or viewpoint changes. The basic architecture of CapsNet used in this study is mentioned in Table 4.

A non-linear squashing function ensures that the output vector length remains between 0 and 1, stabilizing the learning process using Eq. (6):

$$v_j = \frac{\|s_j\|^2}{1 + \|s_j\|^2} \cdot \frac{s_j}{\|s_j\|}, \quad \text{with } s_j = \sum_i c_{ij} \hat{u}_{ji}, \quad (6)$$

where $\hat{u}_{ji} = W_{ij} u_i$ is the prediction vector from capsule i to capsule j , and c_{ij} are the dynamic routing coefficients, as shown in block (6) of Fig. 2. This routing-by-agreement mechanism allows digit capsules to give higher weights to predictions that are consistent across multiple primary capsules, thereby reinforcing correct feature hierarchies.

Despite its structural advantages, CapsNet is highly sensitive to hyperparameters such as learning rate, routing iterations, kernel size, and convolutional filters. Manual selection of these parameters is not only computationally expensive but also result in suboptimal performance, leading to slow convergence, overfitting, or poor generalization.

Algorithm 2 Proposed ROC-Net: Reptile-Optimized Capsule Network

Require: Training data $(X_{\text{train}}, y_{\text{train}})$ with features: *http_method*, *http_uri*, *http_trans_depth*, *http_version*, *http_request_body_len*,, *label*.

- 1: A convolutional layer extracts low-level feature maps.
- 2: Primary capsules group these feature maps into vectors, encoding pose and spatial information.
- 3: Digit capsules aggregate predictions from Primary Capsules using routing-by-agreement, where the vector length indicates the class probability and orientation encodes instantiation parameters.
- 4: A non-linear squashing function ensures capsule output vectors remain between 0 and 1.
- 5: **Input:** Search ranges for hyperparameters, population size N , iterations T
- 6: Initialize population $\{X_1^0, X_2^0, \dots, X_N^0\}$ randomly
- 7: **for** $t = 1$ to T **do**
- 8: **for** each solution X_i^t in population **do**
- 9: Evaluate fitness using CapsNet validation accuracy
- 10: Select peers X_k^t , X_j^t and best solution X_{best}^t
- 11: Update position:

$$X_i^{t+1} = X_i^t + r_1(X_{\text{best}}^t - |r_2 \cdot X_i^t|) + r_3(X_k^t - X_j^t)$$
- 12: **if** convergence phase **then**
- 13: Refine with contraction update:

$$X_i^{t+1} = X_i^t + r_4(X_{\text{best}}^t - X_i^t)$$
- 14: **end if**
- 15: **end for**
- 16: **end for**
- 17: **Output:** Optimal hyperparameter vector X_{best}

Therefore, an effective optimization strategy is essential to fully exploit the representational power of CapsNet in real-world IoT scenarios.

RSA simulates the intelligent hunting strategies of crocodiles, balancing exploration and exploitation in high-dimensional spaces [27]. It starts with a population of candidate solutions (hyperparameter sets), and each solution is updated based on the best-known position and the difference between random peers, using Eq. (7), as shown in Algorithm 2:

$$X_i^{t+1} = X_i^t + r_1(X_{\text{best}}^t - |r_2 \cdot X_i^t|) + r_3(X_k^t - X_j^t) \quad (7)$$

where $r_1, r_2, r_3 \in [0, 1]$ are random coefficients, X_k^t and X_j^t are two randomly selected individuals, and X_{best}^t is the best current solution, as shown in block (6.1) of Fig. 2. In the later stages, RSA employs a contraction behavior to fine-tune solutions around the best candidate [28] using Eq. (8):

$$X_i^{t+1} = X_i^t + r_4 \cdot (X_{\text{best}}^t - X_i^t) \quad (8)$$

This equation refines each candidate solution X_i^t by contracting it toward the current best solution X_{best}^t . The factor r_4 controls the contraction strength, enabling RSA to fine-tune the search around promising regions in the later optimization stages. This helps in refining the hyperparameters while avoiding local optima. When applied to CapsNet, RSA efficiently explored the hyperparameter space and identified an optimal configuration, improving both convergence and accuracy. The hyperparameters of the proposed ROC-Net are tuned using the RSA, and the optimal configuration is summarized in Table 5. The learning rate of 1×10^{-5} is selected, which, although small, ensures stable convergence by preventing overshooting during gradient updates. In the case of the convolutional layer, the best filter count is 32, which is a tradeoff between representational power and computational power. The kernel size of 9 is chosen, which allows the CapsNet to learn

Table 5

Hyperparameter optimization using reptile search algorithm.

Hyperparameters	Search range	Optimal value
Learning rate	[1e-5, 1e-2]	0.00001
Conv1D filters	{32, 64, 128}	32
Kernel size	{3, 5, 7, 9}	9
Routing iterations	{1, 2, 3}	2
Batch size	{32, 64, 128}	64

larger temporal dependencies in the feature space than smaller kernels. The routing iterations that govern the dynamic routing scheme in CapsNet are minimized to 2, which guarantees the attainment of successful feature agreement without the unnecessary computation burden. A batch size of 64 offered the best trade-off between training stability and convergence speed. Together, these optimized hyperparameters enhance the learning ability of CRNet, leading to improved accuracy and generalization while maintaining computational efficiency.

The proposed ROC-Net supports the adaptability of IoT environments better by incorporating RSA-accelerated hyperparameter optimization into CapsNet. IoT networks are by their nature heterogeneous and produce a wide variety of traffic patterns, which complicates the process of IDS. The tuned parameters enable CapsNet to simultaneously learn local and global dependencies in traffic data, train more smoothly, and better generalize to attacks of various types. As a result, the ROC-Net is more robust, more precise, and efficient, which qualifies it to be effective in real-time ID, used in IoT-based systems.

2.6. Proposed Marginal Active-learning with Reptile-optimized Capsule Network (MARCO-Net)

While hyperparameter optimization with RSA significantly improves the convergence and generalization of the ROC-Net, the challenge of data labeling remains unresolved. In real-world IoT environments, obtaining fully labeled datasets is costly, time-consuming, and often impractical, especially when attack samples are scarce or require expert annotation. Thus, even an optimized model like ROC-Net struggle to achieve its full potential if trained only on a small labeled set or on randomly selected instances, which do not always represent the most informative regions of the feature space. This motivates the integration of an active learning strategy to minimize labeling costs while maximizing model performance. Active learning addresses this issue by enabling the model to iteratively select the most informative samples from an unlabeled pool and request their labels. Instead of random sampling, an MBAL is employed, which focuses on samples lying closest to the decision boundary—those where the difference between the top two predicted probabilities is minimal [29]. By querying and labeling these uncertain samples, the model progressively refines its decision boundary using fewer labeled samples.

In the proposed MARCO-Net, MBAL is combined with ROC-Net to combine the representational power of CapsNet, the optimized stability of RSA-driven tuning, and the efficiency of active learning. For a given input x_i , if the two highest predicted class probabilities are p_1 and p_2 , the marginal uncertainty is computed using Eq. (9):

$$\text{Margin}(x_i) = |p_1 - p_2|. \quad (9)$$

Samples with the smallest margins are considered most uncertain and are selected for labeling. In this study, two acquisition iterations are performed, where a total of 8000 uncertain samples are labeled and incrementally added to the training set, as summarized in Algorithm 3. By iteratively refining the labeled dataset, MARCO-Net ensures that ROC-Net learns from the most critical regions of the feature space, reducing redundancy, improving generalization, and significantly lowering labeling requirements. This synergy between optimization and active learning makes MARCO-Net highly effective for IoT intrusion detection, where labeled attack data is limited and costly to obtain. At the

Algorithm 3 Proposed MARCO-Net: Marginal Active-learning with Reptile-optimized Capsule Network

```

1: Input: Dataset  $X$  with features  $http\_method, http\_uri, http\_trans\_depth,$ 
 $http\_version, http\_request\_body\_len, \dots, label$ , query size  $k = 8000$ ,
iterations  $T = 2$ 
2: Split  $X$  into test set, initial labeled set  $L$ , and unlabeled pool  $U$ 
3: for  $t = 1$  to  $T$  do
4:   Train ROC-Net (CapsNet optimized with Reptile-based RSA) on
   current labeled set  $L$ 
5:   Predict class probabilities on unlabeled pool  $U$ 
6:   For each  $x_i \in U$ , compute margin:

    $Margin(x_i) = |p_1(x_i) - p_2(x_i)|$ 

7:   Select query set:

    $QuerySet = \underset{x_i \in U}{\operatorname{argmin}_k} |p_1(x_i) - p_2(x_i)|$ 

8:   Obtain labels for QuerySet and update  $L \leftarrow L \cup QuerySet$ 
9:   Update unlabeled pool  $U \leftarrow U \setminus QuerySet$ 
10: end for
11: Output: Final trained MARCO-Net evaluated on the test set

```

core of CapsNet, the output layer produces vector-lengths representing class probabilities. After each training iteration, the model computes the predicted probabilities on the unlabeled data, and the samples with the smallest margin values are selected using Eq. (10):

$$QuerySet = \underset{x_i \in X_{\text{unlabeled}}}{\operatorname{argmin}_k} |p_1(x_i) - p_2(x_i)| \quad (10)$$

where k most uncertain samples from the unlabeled set $X_{\text{unlabeled}}$ by computing the margin $|p_1(x_i) - p_2(x_i)|$, where $p_1(x_i)$ and $p_2(x_i)$ represent the highest and second-highest predicted class probabilities for sample x_i . A smaller margin indicates higher ambiguity, so the $k = 8000$ samples with the lowest margins are added to the QuerySet for annotation. This query set is then labeled and merged into the training set, gradually improving the model's decision boundary with minimal labeled data. In this way, MARCO-Net effectively combines the representation power of ROC-Net with the data efficiency of active learning to yield strong performance with fewer labeled samples. Through this iterative process, MARCO-Net not only reduces the dependency on large fully labeled datasets but also ensures that the most informative samples contribute to model training. The model identifies and optimizes generalization by focusing on uncertain cases that are closer to the decision boundary, reduces labeling costs, and speeds up convergence. Therefore, MARCO-Net is especially useful with IoT IDS where labeled attacks are usually limited and costly to acquire. Finally, a combination of MBAL and optimized CapsNet architecture creates a strong, and data-efficient solution to the security of complex IoT environments.

3. Results and discussion

The performance of the existing and proposed models is evaluated using multiple classification measures which provides the mathematical definitions of several metrics. All these measures provide a thorough analysis of predictive performance, reliability, and error analysis. Table 6 also summarizes the major insights and it is clear that the main limitations of baseline models are highlighted and the advantages of the proposed ROC-Net and MARCO-net are also presented. Although the traditional models have generalization, instability, and computational-inefficiency problems, the proposed models exhibit better generalization, robustness, and practical-applicability, with MARCO-Net being the most accurate and efficient of all.

The Table 7 presents the key performance measures and their corresponding formulas used to evaluate the performance of classification

Table 6

Accuracy insights on existing models and proposed models.

Model (Accuracy)	Insights
DT 0.62	Highly prone to overfitting on training data and unable to capture complex feature interactions.
ETC 0.50	Computationally inefficient with increasing data size and often produces unstable predictions due to randomness.
Node 0.80	Requires significant computational resources and suffers from long training times, reducing scalability.
DBN 0.69	Difficult to train because of vanishing gradients and highly sensitive to hyperparameter initialization.
FT-Transformer 0.78	Performs well on tabular data but requires extensive tuning and large training time. Its attention-based structure struggles when feature importance varies significantly across samples.
CNN+LSTM 0.75	Captures spatial-temporal patterns but is computationally heavy. Performance degrades when temporal dependencies are weak, and training becomes unstable on large feature spaces.
CapsNet 0.74	Very computationally expensive and challenging to optimize for large-scale datasets.
ROC-Net 0.87	Provides balanced performance across all evaluation measures and ensures reliable classification even in complex scenarios. Additionally, it offers strong generalization ability, making it robust against overfitting.
MARCO-Net 0.90	Achieves the highest predictive performance with superior accuracy and efficiency. Moreover, it combines robustness with faster execution, making it highly practical for real-world applications.

Table 7

Performance measures and formulas.

Measures	Formula
1. Accuracy	$\frac{TP+TN}{TP+TN+FP+FN} \times 100$
2. Precision	$\frac{TP}{TP+FP} \times 100$
3. Recall	$\frac{TP}{TP+FN} \times 100$
4. F1-score	$2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \times 100$
5. MCC	$\frac{TP \times TN - FP \times FN}{\sqrt{(TP+FP)(TP+FN)(TN+FP)(TN+FN)}}$
6. Log loss	$-\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$
7. Hamming loss	$\frac{1}{N} \sum_{i=1}^N 1(y_i \neq \hat{y}_i)$

models. The terms True Positive (TP), True Negative (TN), FP, and False Negative (FN) are fundamental to these metrics:

- **TP:** The number of instances correctly predicted as positive. These are the cases where the model correctly identifies the positive class.
- **TN:** The number of instances correctly predicted as negative. These are the cases where the model correctly identifies the negative class.
- **FP:** The number of instances incorrectly predicted as positive. These are the cases where the model mistakenly classifies a negative instance as positive, also known as a “type I error”.
- **FN:** The number of instances incorrectly predicted as negative. These are the cases where the model mistakenly classifies a positive instance as negative, also known as a “type II error”.

These values are used to calculate performance metrics which provide insights into the model's ability to make correct predictions across various scenarios.

Table 8
Dataset Splitting for Active Learning.

Set	Sample size	% of total dataset	Purpose
Test set	13,014	20%	For final evaluation of the model
Labeled set	26,028	40%	For training the model
Unlabeled set	26,028	40%	For active learning (iterative labeling)

Table 9
Performance comparison of existing models, ROC-Net, and MARCO-Net.

Metric	DT	ETC	Node	DBN	FT-Transformer	CNN+LSTM	CapsNet	ROC-Net	MARCO-Net
Accuracy	0.62	0.50	0.80	0.69	0.78	0.75	0.74	0.87	0.90
Precision	0.59	0.50	0.78	0.69	0.78	0.85	0.86	0.91	0.89
Recall	0.81	0.69	0.84	0.69	0.57	0.59	0.57	0.85	0.92
F1-score	0.68	0.66	0.81	0.76	0.72	0.70	0.68	0.87	0.90
MCC	0.27	0.34	0.67	0.40	0.63	0.52	0.51	0.75	0.81
Cohen's Kappa	0.43	0.25	0.66	0.40	0.57	0.49	0.48	0.75	0.81
Hamming loss	0.37	0.49	0.19	0.49	0.21	0.27	0.25	0.12	0.09
Log loss	0.65	0.69	0.67	0.66	0.51	0.67	2.90	0.70	0.52
Execution time (s)	0.15	0.25	132	20.4	416.76	143.14	385.75	99.64	2.09

3.1. Experimental setup and data splitting

All experiments are performed on Google Colab for model development and training. The computational setup used for execution is a machine running Windows 10, powered by an Intel(R) Core(TM) i5-7300U processor operating at 2.60 GHz (2.71 GHz). Multiple simulations are performed, and the best results for each model configuration are reported in this section. The dataset is divided into 80% for training and 20% for testing to evaluate the baseline and proposed models performance on unseen data. In the case of MARCO-Net, the data is first split into 20% as the test set and the remaining 80% is further divided equally into an initial labeled training set and an unlabeled pool as shown in Table 8.

3.1.1. Comparative analysis of results

A comparative evaluation of the models as shown in Fig. 3 and Table 9 demonstrates that the proposed architectures significantly outperform the existing approaches, with MARCO-Net achieving the highest accuracy of 0.90 and ROC-Net following with 0.87, surpassing all baseline models including Node at 0.80, FT-Transformer at 0.78, which leverages attention-based tabular feature modeling, and CNN+LSTM at 0.75, which captures spatial and temporal dependencies in IoT traffic. Precision results further reinforce this trend, where ROC-Net records the highest value of 0.91, followed closely by MARCO-Net at 0.89 and CapsNet at 0.86, while CNN+LSTM and FT-Transformer achieve 0.85 and 0.78 respectively, benefiting from sequential processing and feature-wise attention to reduce false alarms. In terms of recall, MARCO-Net attains the top value of 0.92, followed by ROC-Net at 0.85 and Node at 0.84, whereas CapsNet, FT-Transformer, and CNN+LSTM register lower recall due to their limited ability to capture minority-class attack patterns. Overall, the significant improvements achieved by ROC-Net and MARCO-Net highlight the effectiveness of RSA-driven optimization in stabilizing capsule representations and the strength of margin-based active learning in refining decision boundaries, enabling more reliable and robust intrusion detection across diverse IoT traffic distributions. The F1-score harmonizes precision and recall into a single indicator of balanced performance. MARCO-Net attains the highest F1-score with 0.90, followed by ROC-Net with 0.87, Node with 0.81, DBN with 0.76, FT-Transformer with 0.72 and CNN+LSTM with 0.70, where FT-Transformer's transformer blocks help achieve moderate balance despite lower recall, and CNN+LSTM maintains stability through hybrid spatial-temporal learning. DT, ETC, and CapsNet each at 0.68, 0.66, and 0.68. The combined improvement in both precision and recall for the proposed models translates into the best F1-scores, confirming that they not only reduce false alarms but also catch a larger share of true attacks, which is the desired operating point for IDS in IoT settings.

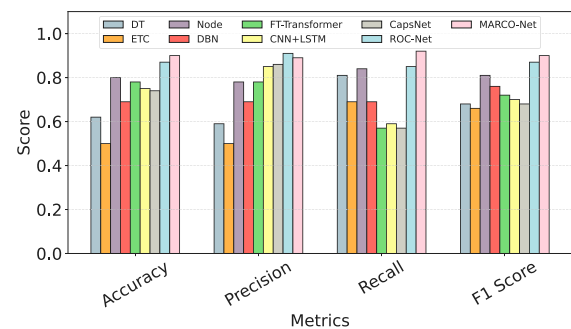


Fig. 3. Performance comparison of different metrics.

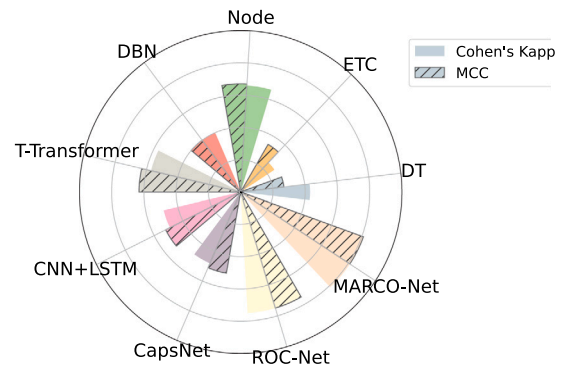


Fig. 4. Cohen Kappa and MCC comparison.

3.1.2. Analysis of MCC and Cohen's Kappa

The reliability of all models is assessed using MCC and Cohen's Kappa, and the results demonstrate a consistent advantage for the proposed architectures, as Fig. 4 shows. MARCO-Net achieves the highest values with 0.81 for both metrics, followed by ROC-Net with 0.75, reflecting stronger agreement with ground truth due to optimized capsule routing and margin-based active learning. Among the baselines, NODE performs best with MCC of 0.67 and Kappa of 0.66, while FT-Transformer shows moderate reliability with MCC of 0.63 and Kappa of 0.57, and CNN+LSTM follows with 0.52 for both metrics owing to its capacity to learn temporal-spatial patterns despite sensitivity to noisy IoT traffic. CapsNet records lower agreement with MCC of 0.51 and Cohen's Kappa of 0.48 due to calibration issues, whereas DBN reaches 0.40 for both measures because of instability under heterogeneous traffic. DT and ETC show the weakest reliability, with DT achieving MCC

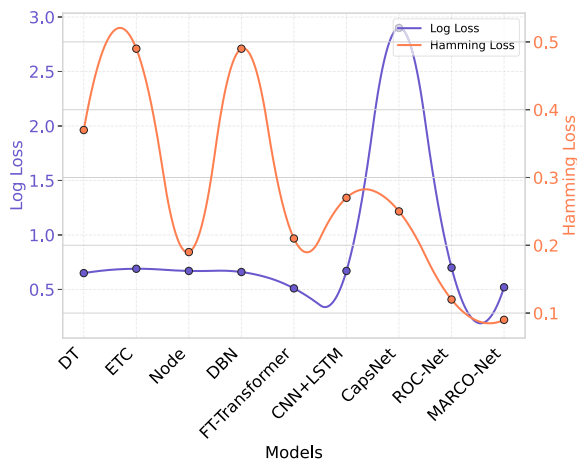


Fig. 5. Log loss and Hamming loss comparison.

of 0.27 and Cohen's Kappa of 0.43, and ETC yielding 0.34 and 0.25, reflecting overfitting and underfitting tendencies respectively. Overall, the results indicate that the proposed ROC-Net and MARCO-Net deliver substantially superior agreement-aware performance compared to both classical and DL baselines in IoT intrusion detection.

3.1.3. Analysis of loss and log loss

The comparison of error- and calibration-based metrics in Fig. 5 shows that MARCO-Net achieves the most reliable predictions with the lowest Hamming loss of 0.09 and log loss of 0.52, followed by ROC-Net with 0.12 and 0.70, respectively. Among the baseline models, NODE performs comparatively better with a Hamming loss of 0.19 and log loss of 0.67, while CapsNet records higher errors with values of 0.25 and 2.90. The Hamming loss and log loss of FT-Transformer are 0.21 and 0.51 respectively, which is a result of attention-based learning of calibration, and CNN+LSTM shows a loss of 0.27 and 0.67, respectively, which is an average performance determined by its hybrid temporal-spatial model. Other baselines, including DBN, DT, and ETC, show higher error rates and poorer calibration, demonstrating their limitations in handling diverse IoT traffic patterns. The notable reduction in both losses for MARCO-Net and ROC-Net highlights the advantage of RSA optimization and margin-based active learning, which not only lowers misclassification errors but also improves probability calibration, ultimately yielding the most stable and trustworthy predictions.

3.1.4. Analysis of execution time

Execution time is analyzed to assess the computational efficiency of the proposed and baseline models, reflecting the impact of architectural complexity and optimization strategies. Fig. 6 presents the execution time comparison, showing significant variations among models. DT records the lowest execution time of 0.15 s due to its simple tree structure, followed by ETC at 0.25 s as its randomization and multiple splits slightly increase computation. Node requires 132 s because of its deeper ensemble architecture, while DBN takes 20.40 s owing to layer-wise pretraining despite its shallower structure. CNN+LSTM achieves 143.14 s, combining CNN for spatial feature extraction and LSTM for temporal learning. CapsNet requires 385.75 s as dynamic routing between capsules is computationally intensive. FT-Transformer records 416.76 s due to the attention mechanism processing long sequences. ROC-Net reduces this to 99.64 s by using RSA for hyperparameter tuning, and the proposed MARCO-Net achieves a remarkably low execution time of 2.09 s by applying active learning to focus on the most uncertain samples, significantly lowering computational cost while maintaining high predictive performance.

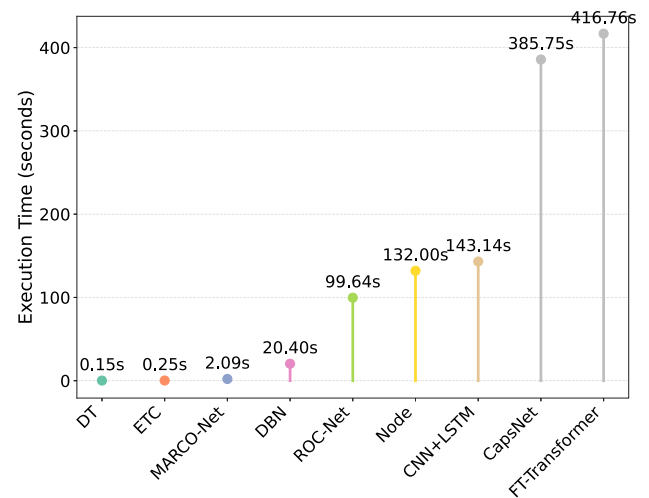


Fig. 6. Execution time comparison.

3.2. Comparative analysis with different data balancing techniques

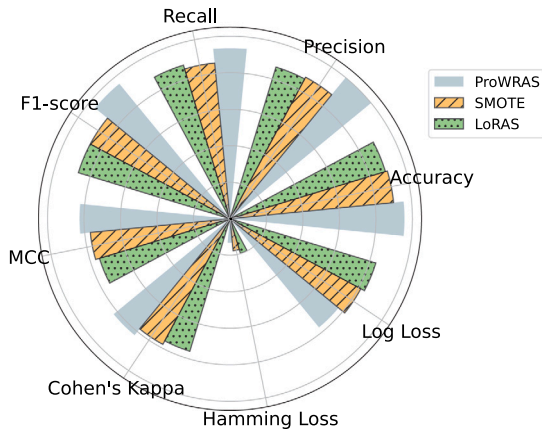
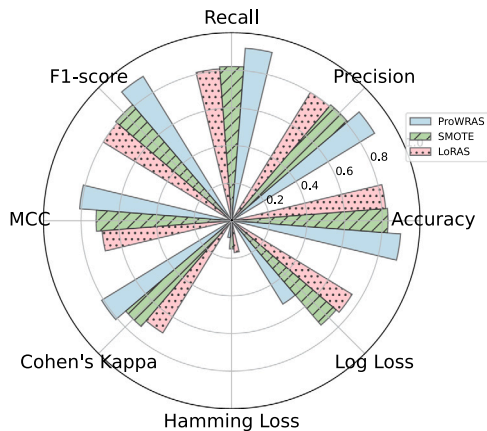
The comparative results reported in Table 10 and Fig. 7 highlight the superiority of the proposed ProWRAS balancing technique when integrated with both ROC-Net and MARCO-Net, compared to conventional oversampling strategies such as SMOTE [30] and Localized Random Affine Shadow Sampling (LoRAS) [31]. To clearly demonstrate the effect of the implemented data balancing methods, Table 11 shows the number of samples in each class before and after balancing. The original data is highly skewed with the normal class of 32,535 samples and the attack class with 17,465 samples. Following the implementation of ProWRAS, SMOTE, and LoRAS, all the methods were able to balance the dataset and generated synthetic samples of the minority class, which gave equal representation of 32,535 samples per class.

Specifically, when combined with ProWRAS, ROC-Net achieves an accuracy of 0.87, precision of 0.91, recall of 0.85, and F1-score of 0.87. These values are significantly higher than those obtained with SMOTE, which records an accuracy of 0.82 and F1-score of 0.79 but often generates overlapping synthetic samples that reduce class separability, and LoRAS, which yields an accuracy of 0.81 and F1-score of 0.79 yet suffers from high computational cost due to its reliance on localized Gaussian noise sampling. For MARCO-Net, integration with ProWRAS achieves the best overall results, yielding an accuracy of 0.90, recall of 0.92, precision of 0.89, and an F1-score of 0.90. Compared to SMOTE with accuracy of 0.83, recall of 0.82, and F1-score of 0.81, which often generates synthetic samples too close to class boundaries leading to overlapping and noise, and LoRAS with accuracy of 0.82, recall of 0.81, and F1-score of 0.80, which incurs higher computational overhead due to localized Gaussian sampling, ProWRAS demonstrates superior effectiveness in enhancing generalization. The high recall of 0.92 indicates that MARCO-Net with ProWRAS is particularly effective in correctly detecting attack instances, which is critical in IDS tasks where FNs are more detrimental than FPs. The robustness of the models is validated through correlation-based metrics. For ROC-Net with ProWRAS, MCC and Cohen's Kappa both reach 0.75, whereas MARCO-Net achieves 0.81 for both metrics. In contrast, MCC values under SMOTE and LoRAS are limited to 0.70 and 0.68, respectively, while Cohen's Kappa remains lower at 0.72 and 0.70. This is supported by error-based metrics. Hamming loss at ProWRAS is reduced to 0.12 in ROC-Net and 0.09 in MARCO-Net, which is low compared to SMOTE (0.16 and 0.15) and LoRAS (0.18 and 0.17). Similarly, log loss is reduced to 0.70 and 0.52 under ProWRAS, compared to 0.74/0.73 with SMOTE and 0.76/0.75 with LoRAS. The lower log loss demonstrates greater confidence in predictions, with MARCO-Net exhibiting the most

Table 10

Performance comparison of ROC-Net and MARCO-Net with different data balancing techniques.

Metric	ProWRAS		SMOTE		LoRAS	
	ROC-Net	MARCO-Net	ROC-Net	MARCO-Net	ROC-Net	MARCO-Net
Accuracy	0.87	0.90	0.82	0.83	0.81	0.82
Precision	0.91	0.89	0.80	0.81	0.79	0.80
Recall	0.85	0.92	0.78	0.82	0.80	0.81
F1-score	0.87	0.90	0.79	0.81	0.79	0.80
MCC	0.75	0.81	0.70	0.72	0.68	0.69
Cohen's Kappa	0.75	0.81	0.71	0.74	0.69	0.70
Hamming loss	0.12	0.09	0.16	0.15	0.18	0.17
Log loss	0.70	0.52	0.74	0.73	0.76	0.75
Execution time (s)	99.64	2.09	98.51	2.05	97.90	2.00

**Fig. 7.** Performance comparison of ROC-Net on different balancing techniques.**Fig. 8.** Performance comparison of MARCO-Net on different balancing techniques.

reliable probability estimations. There is another interesting outcome that is gained through execution time. Although ProWRAS has shown higher predictive performance, the computational efficiency is similar to balancing techniques. ROC-Net takes about 99.64 s with ProWRAS, as compared to 98.51 s with SMOTE and 97.90 s with LoRAS. MARCO-Net is incredibly efficient, taking approximately 2.09 s with ProWRAS, almost equal to 2.05 s and 2.00 s with SMOTE and LoRAS. This confirms that the improved performance gains are not achieved at the expense of additional computational overhead. Overall, the results in Table 10 confirm that integrating ProWRAS with ROC-Net and MARCO-Net yields significant performance gains (see Fig. 8).

Table 11

Class distribution before and after applying ProWRAS, SMOTE, and LoRAS.

Condition	Normal	Attack
Original data	32,535	17,465
After balancing (ProWRAS)	32,535	32,535
After balancing (SMOTE)	32,535	32,535
After balancing (LoRAS)	32,535	32,535

Table 12

Performance using different active learning strategies.

Metric	MARCO-Net	EBAL	UBAL
Accuracy	0.90	0.84	0.83
Precision	0.89	0.81	0.80
Recall	0.92	0.83	0.82
F1-score	0.90	0.82	0.81
MCC	0.81	0.73	0.71
Cohen's Kappa	0.81	0.74	0.72
Hamming loss	0.09	0.16	0.17
Log loss	0.52	0.71	0.74
Execution time (s)	2.09	2.05	2.03

3.3. Comparative analysis with different active learning strategies

The results in Table 12 and Fig. 9 clearly demonstrate the superiority of the proposed MARCO-Net compared to Entropy-based Active Learning (EBAL) [32] and Uncertainty-based Active Learning (UBAL) [33]. Our proposed MARCO-Net achieves the highest accuracy of 0.90, surpassing EBAL with 0.84 and UBAL with 0.83. This indicates that the MBAL strategy in MARCO-Net selects more informative and representative samples, thereby enhancing the CapsNet ability to generalize across unseen data. In terms of precision, MARCO-Net attains a value of 0.89, which is significantly higher than EBAL with 0.81 and UBAL with 0.80. The relatively lower precision of EBAL stems from its reliance on entropy-based selection, which may overemphasize uncertain but uninformative samples, while UBAL suffers from focusing on uncertainty alone, often querying redundant samples that provide limited additional knowledge. This improvement highlights the effectiveness of MARCO-Net in minimizing FPs, which is a critical factor in IDS where benign traffic must not be mistakenly flagged as malicious. MARCO-Net achieves a recall of 0.92, compared to 0.83 for EBAL and 0.82 for UBAL, illustrating its strong ability to correctly identify malicious instances and minimize FNs, an essential aspect for IDS as missed attacks can have severe consequences. The F1-score, which balances precision and recall, is also maximized by MARCO-Net at 0.90, in contrast to 0.82 and 0.81 for EBAL and UBAL, respectively. This balance confirms that MARCO-Net maintains both reliability and robustness in its predictions.

Correlation-based measures further validate the effectiveness of the proposed model. MARCO-Net achieves an MCC of 0.81 and a Cohen's Kappa of 0.81, both outperforming EBAL with 0.73 and 0.74 and UBAL with 0.71 and 0.72. The lower scores of EBAL highlight its tendency to query noisy samples due to entropy maximization, whereas UBAL's simplistic uncertainty-driven selection reduces diversity in the labeled

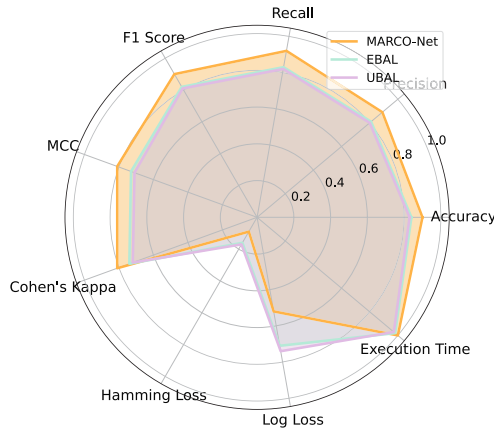


Fig. 9. Performance comparison of ROC-Net and MARCO-Net on different active learning techniques.

Table 13
Performance comparison with different optimization techniques.

Metric	BO	AO	ROC-Net
Accuracy	0.84	0.86	0.87
Precision	0.88	0.89	0.91
Recall	0.81	0.83	0.85
F1-score	0.84	0.86	0.87
MCC	0.71	0.73	0.75
Cohen's Kappa	0.71	0.73	0.75
Hamming loss	0.16	0.13	0.12
Log loss	0.75	0.73	0.70
Execution time (s)	101.25	100.18	99.64

set, weakening overall agreement across classes. Error-based metrics reinforce these findings: MARCO-Net reduces the Hamming loss to 0.09, compared to 0.16 and 0.17 for EBAL and UBAL, respectively. Similarly, MARCO-Net minimizes log loss to 0.52, indicating higher confidence in its probabilistic predictions, whereas EBAL and UBAL strategies exhibit less reliable estimates with values of 0.71 and 0.74. Despite these substantial gains in predictive performance, MARCO-Net maintains competitive computational efficiency, requiring 2.09 s of execution time, which is nearly identical to EBAL with 2.05 s and UBAL with 2.03 s. This demonstrates that the performance improvements of MARCO-Net are not achieved at the expense of computational overhead. Overall, the results confirm that the proposed MBAL strategy significantly enhances detection capability and decision reliability for IDS in IoT environments, while addressing the key shortcomings of EBAL and UBAL by avoiding noisy and redundant queries.

3.4. Comparative analysis with different hyperparameter optimization techniques

The results presented in Table 13 and Fig. 10 highlight the comparative performance of Bayesian Optimization (BO) [34], Aquila Optimization (AO) [35], and the RSA when applied to CapsNet. BO exhibits the weakest performance across most metrics, with an accuracy of 0.84, precision of 0.88, recall of 0.81, and F1-score of 0.84. The relatively low recall shows that BO misses a substantial proportion of TPs, reducing the overall balance between sensitivity and specificity. This shortcoming arises because BO relies on surrogate models that are effective in smooth and low-dimensional search spaces but struggle in the highly non-convex and irregular hyperparameter landscape of CapsNet. As a result, BO often converges prematurely to suboptimal configurations, reducing generalization performance. This limitation is further reflected by its lower MCC of 0.71 and Cohen's Kappa of 0.71, which reflect weaker agreement with ground truth compared to the other methods.

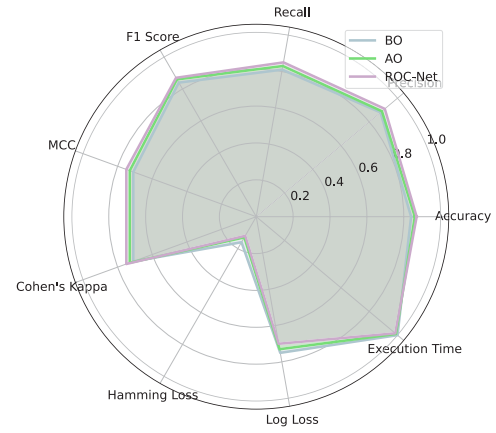


Fig. 10. Performance comparison of ROC-Net and MARCO-Net on different optimization techniques.

AO improves over BO by achieving an accuracy of 0.86, precision of 0.89, recall of 0.83, and F1-score of 0.86. The improvement in recall and precision suggests that AO is better at balancing exploration and exploitation in the hyperparameter space. However, AO shows instability, as reflected by its moderate MCC and Cohen's Kappa values of 0.73, and its log loss of 0.73, which indicates that the model is still prone to miscalibration in probabilistic predictions. AO's main limitation is its tendency to oversample exploratory regions, causing fluctuations in convergence behavior and preventing fine-grained refinement of CapsNet's parameters. Consequently, although AO surpasses BO, it fails to consistently optimize the model to its full potential. The ROC-Net delivers the best overall results, with an accuracy of 0.87, precision of 0.91, recall of 0.85, and F1-score of 0.87. These results reflect a stronger balance between sensitivity and precision, directly addressing the weaknesses observed in BO and AO. RSA's meta-learning mechanism enables adaptive updates by leveraging gradient-based optimization across multiple tasks, which allows it to escape poor local minima and refine parameters in a more robust manner. This results in higher MCC and Cohen's Kappa values of 0.75, indicating stronger reliability in classification. Furthermore, ROC-Net achieves the lowest log loss of 0.70 and the lowest Hamming loss of 0.12, highlighting its ability to produce well-calibrated predictions with fewer misclassifications. The slight reduction in execution time 99.64 s compared to 101.25 s for BO and 100.18 s for AO also shows that ROC-Net maintains efficiency while delivering higher predictive performance. Overall, the empirical results confirm that RSA is more effective for hyperparameter tuning of CapsNet compared to BO and AO techniques.

3.5. Discussion on MARCO-Net performance across different active learning settings

As the experiment of MARCO-Net using various active learning settings reveals, query size has a direct effect on predictive behavior, model stability, and error nature. With a query size of 5000 and 2 iterations, the model attains an accuracy of 0.87, precision and recall of 0.88 and 0.87 respectively as indicated in Table 14. These findings suggest that the moderate size of query enables MARCO-Net focus on a sample that is highly informative without overloading the learning process with borderline cases. The resultant F1-score of 0.86 and the MCC and Cohen Kappa values of 0.79 are additional indications that the model is well aligned with the ground truth, in addition to balancing FPs and FNs. The low Hamming Loss of 0.11 and stable log loss at 0.67 indicate that MARCO-Net is at a good calibration and has minimal misclassification under this configuration, and also demands a small execution time of 1.66, which demonstrates the process to be efficient and reliable.

Table 14

Performance comparison for different active learning iterations and query sizes.

Metric	Iter = 2, Query = 5000	Iter = 2, Query = 10000
Accuracy	0.87	0.75
Precision	0.88	0.67
Recall	0.87	0.97
F1 score	0.86	0.79
MCC	0.79	0.55
Cohen's Kappa	0.79	0.50
Hamming loss	0.11	0.25
Log loss	0.67	0.66
Execution time (s)	1.66	1.70

As the query size is raised up to 10000, the model becomes more susceptible to the detection of attack instances as indicated by the significant improvement in the recall to 0.97. This is however at the expense of accuracy which reduces to 0.67, meaning that there would be more false alarms. The accuracy also decreases to 0.75, which indicates that the greater the number of unconfident samples, the higher the noise that is added to the learning process. Though still a very reasonable F1-score of 0.79, both MCC and Cohen Kappa are much lower (0.55 and 0.50 respectively) and indicate less consistency and agreement with the actual labels. The Hamming Loss increases to 0.25, indicating a higher misclassification rate, while log loss remains nearly unchanged at 0.66. The execution time also remains stable at 1.70 s, showing that computational cost is not significantly affected by the query size.

These observations suggest that increasing the query sizes can increase the detection capabilities of the model on the difficult instances, but it can also bring some instability to the performance and false positives. The 5000 queries, two-iteration configuration shows a more balanced and predictable performance profile, maintaining accuracy, stability, and calibration. As a result, the choice of a query size is essential in facilitating the effective functioning of MARCO-Net under active learning conditions.

3.6. 10-fold cross-validation

A 10-FCV strategy is used in order to guarantee the viability and overallizability of the proposed IDS models [36]. The data is split into ten equal subsets and one of them is used once during the testing and the rest of the subsets are used during training [37]. This is done ten times and the findings are averaged to produce a complete performance estimate. The benefit of 10-FCV is that it reduces the biasing effects of a single train-test split and to give a more consistent assessment of the models on different cross-datasets splits. This confirmation is especially crucial in the tasks of IDS when the imbalance of classes and variations in features may otherwise result in unreliable results.

Table 15 presents the detailed 10-FCV results of ROC-Net and MARCO-Net across multiple performance metrics. The average accuracy in the folds was 0.868 in ROC-Net, and 0.900 in MARCO-Net. This enhancement shows that the incorporation of integrating MBAL into CapsNet improves the overall reliability of the classification. Precision values indicate that ROC-Net is marginally better than MARCO-Net with averages of 0.905 than 0.890. This indicates that ROC-Net generates fewer FPs, although MARCO-Net balances this by excelling in recall, where it records 0.920 compared to ROC-Net's 0.845. The increased recall of MARCO-Net indicates its greater capacity to accurately recognize instances of attacks, which is essential in IDS.

F1-score, a trade-off between precision and recall, also demonstrates that MARCO-Net is superior with an average of 0.900 as compared to ROC-Net whose average is 0.867. Similarly, correlation-based measures such as MCC and Cohen's Kappa show consistent improvements for MARCO-Net, both achieving averages of 0.810 compared to 0.745 for ROC-Net. These findings underscore a more dependable consensus of

predictions and ground truth in case of active learning being incorporated. These findings are further verified by error-based metrics. MARCO-Net offers Hamming loss of 0.090 and log loss of 0.522; in comparison ROC-Net offers 0.125 and 0.708 respectively. These decreases show that MARCO-Net is not only enhancing detection rates, but also confidence in classification outputs.

In terms of computational efficiency, a huge difference is observed between the two models. ROC-Net requires an average execution time of 99.46 s, while MARCO-Net drastically reduces this to only 2.092 s. This underscores the feasibility of MARCO-Net, which has better predictive precision and reduced error rates at no incremental computational expenses. In general, Table 15 verify that MARCO-Net provide a better balanced and effective performance of precision and recall, providing a reliable attack detection in the IoT environment, and are also computationally more efficient than ROC-Net.

3.7. Confidence interval analysis of ROC-Net and MARCO-Net

Confidence intervals specify a statistical interval whereby the actual value of a measure is likely to fall under a fixed confidence level, usually 95 percent. These measures give an analysis of the dependability and soundness of the approximated performance of a model by taking into account fluctuation among various folds of a cross-evaluation. A smaller confidence interval represents more stability and consistency, and a broader interval represents more variability within subsets.

The results in Table 16 show the comparative performance of ROC-Net and MARCO-Net under 10-FCV with 95% confidence intervals. Compared to MARCO-Net, which has a higher stability of accuracy and a narrower confidence interval of (0.8942, 0.9058), ROC-Net, in terms of accuracy, has a mean of 0.866 and a confidence interval of (0.8591, 0.8729), implying a greater stability and accuracy in classification. Precision for ROC-Net is slightly higher at 0.905 compared to 0.890 for MARCO-Net, showing that ROC-Net produces fewer false alarms. However, MARCO-Net exhibits considerably higher recall of 0.920 than 0.845 in ROC-Net, indicating that MARCO-Net identifies a higher percentage of real attacks. The resultant trade-off gives a balanced F1-score of 0.900 to MARCO-Net, a little better than the 0.865 of ROC-Net.

The MCC and Cohen's Kappa both stand at 0.745 for ROC-Net, improving to 0.810 for MARCO-Net. This indicates stronger agreement between predicted and actual labels in MARCO-Net, particularly in imbalanced data scenarios. Error-based measures further emphasize MARCO-Net's superiority. Hamming loss decreases from 0.125 in ROC-Net to 0.090 in MARCO-Net, while log loss reduces from 0.708 to 0.524, demonstrating that MARCO-Net not only makes more correct predictions but also produces better-calibrated probability estimates. Another area of difference is the execution time where ROC-Net takes 99.359 s and MARCO-Net takes only 2.092 s to train and evaluate which demonstrates the computational efficiency of the active learning strategy. In general, despite ROC-Net showing high levels of precision and balanced performance, MARCO-Net has a significantly higher recall, F1-score, MCC, Cohen Kappa and error based metrics, at a reduced computational cost. It renders the MARCO-Net an extremely practical and trustworthy paradigm of IDS in IoT settings.

3.8. Statistical validation using Wilcoxon Signed-Rank Test

The Wilcoxon Signed-Rank Test is a non-parametric statistical method designed to compare two paired samples in order to determine whether their population mean ranks differ [38]. It is particularly effective when the data does not follow a normal distribution, making it well-suited for evaluating DL models across different metrics. The test works by first calculating the difference between paired observations, discarding zero differences, and then ranking the absolute values of the differences. Each rank is then assigned the sign corresponding to

Table 15

10-fold cross-validation performance of ROC-Net and MARCO-Net.

Metric	Model	Fold1	Fold2	Fold3	Fold4	Fold5	Fold6	Fold7	Fold8	Fold9	Fold10	Average
Accuracy	ROC-Net	0.87	0.86	0.88	0.86	0.87	0.85	0.88	0.86	0.87	0.86	0.86
	MARCO-Net	0.90	0.89	0.91	0.90	0.89	0.91	0.90	0.90	0.89	0.91	0.90
Precision	ROC-Net	0.91	0.90	0.92	0.90	0.91	0.89	0.91	0.90	0.91	0.90	0.90
	MARCO-Net	0.89	0.88	0.90	0.89	0.88	0.90	0.89	0.89	0.88	0.90	0.89
Recall	ROC-Net	0.85	0.84	0.86	0.84	0.85	0.83	0.85	0.84	0.85	0.84	0.84
	MARCO-Net	0.92	0.91	0.93	0.92	0.91	0.93	0.92	0.92	0.91	0.93	0.92
F1-score	ROC-Net	0.87	0.86	0.88	0.86	0.87	0.85	0.87	0.86	0.87	0.86	0.86
	MARCO-Net	0.90	0.89	0.91	0.90	0.89	0.91	0.90	0.90	0.89	0.91	0.90
MCC	ROC-Net	0.75	0.74	0.76	0.74	0.75	0.73	0.75	0.74	0.75	0.74	0.74
	MARCO-Net	0.81	0.80	0.82	0.81	0.80	0.82	0.81	0.81	0.80	0.82	0.81
Cohen's Kappa	ROC-Net	0.75	0.74	0.76	0.74	0.75	0.73	0.75	0.74	0.75	0.74	0.74
	MARCO-Net	0.81	0.80	0.82	0.81	0.80	0.82	0.81	0.81	0.80	0.82	0.81
Hamming loss	ROC-Net	0.12	0.13	0.11	0.13	0.12	0.14	0.12	0.13	0.12	0.13	0.12
	MARCO-Net	0.09	0.10	0.08	0.09	0.10	0.08	0.09	0.09	0.10	0.08	0.09
Log loss	ROC-Net	0.70	0.72	0.68	0.72	0.71	0.73	0.70	0.71	0.70	0.71	0.70
	MARCO-Net	0.52	0.54	0.50	0.53	0.54	0.51	0.52	0.53	0.54	0.51	0.52
Execution time (s)	ROC-Net	99.64	98.12	101.20	97.89	99.32	100.01	98.74	99.58	100.23	98.86	99.46
	MARCO-Net	2.09	2.11	2.07	2.10	2.09	2.08	2.10	2.09	2.11	2.08	2.092

Table 16

Results with 95% confidence intervals for ROC-Net and MARCO-Net.

Metric	ROC-Net (Mean $\pm$ 95% CI)		MARCO-Net (Mean $\pm$ 95% CI)	
Accuracy	0.866	(0.8591, 0.8729)	0.900	(0.8942, 0.9058)
Precision	0.905	(0.8989, 0.9111)	0.890	(0.8842, 0.8958)
Recall	0.845	(0.8389, 0.8511)	0.920	(0.9142, 0.9258)
F1-score	0.865	(0.8589, 0.8711)	0.900	(0.8942, 0.9058)
MCC	0.745	(0.7389, 0.7511)	0.810	(0.8042, 0.8158)
Cohen's Kappa	0.745	(0.7389, 0.7511)	0.810	(0.8042, 0.8158)
Hamming loss	0.125	(0.1189, 0.1311)	0.090	(0.0842, 0.0958)
Log loss	0.708	(0.6980, 0.7180)	0.524	(0.5138, 0.5342)
Execution time (s)	99.359	(98.6426, 100.0754)	2.092	(2.0826, 2.1014)

Table 17

Wilcoxon Signed-Rank Test between CapsNet and proposed models.

Metric	Comparison	W-Statistic	p-value
Accuracy	CapsNet vs. ROC-Net	0.0	0.005
	CapsNet vs. MARCO-Net	0.0	0.005
Precision	CapsNet vs. ROC-Net	1.0	0.011
	CapsNet vs. MARCO-Net	2.0	0.017
Recall	CapsNet vs. ROC-Net	0.0	0.005
	CapsNet vs. MARCO-Net	0.0	0.005
F1-score	CapsNet vs. ROC-Net	1.0	0.011
	CapsNet vs. MARCO-Net	1.0	0.011
MCC	CapsNet vs. ROC-Net	1.0	0.011
	CapsNet vs. MARCO-Net	1.0	0.011

the original difference. The test statistic is obtained by summing these signed ranks, which is denoted in Eq. (11):

$$W = \sum_{i=1}^n \text{sign}(d_i) \cdot R_i \quad (11)$$

where d_i represents the difference between the paired values for the i th observation, and R_i denotes the rank of the absolute difference. A smaller value of the test statistic W indicates a stronger difference between the two compared models. The significance of the results is evaluated using the p -value. If the p -value is less than the chosen significance threshold, typically 0.05, the null hypothesis of no difference between the models is rejected. Table 17 reports the results of the Wilcoxon Signed-Rank Test comparing the baseline CapsNet with the proposed ROC-Net and MARCO-Net across different performance metrics. For accuracy, the comparison between CapsNet and both ROC-Net and MARCO-Net yields a W-statistic of 0.0 with a p -value of 0.005, indicating a statistically significant improvement for both proposed

models. For precision, the comparison shows W-statistics of 1.0 and 2.0 with p -values of 0.011 and 0.017, respectively, showing that although both models outperform CapsNet, the gains in precision are slightly less pronounced than in accuracy. In terms of recall, the comparisons once again show strong significance with W-statistics of 0.0 and p -values of 0.005 for both models, reinforcing their ability to correctly identify attack instances with higher reliability than CapsNet.

For the F1-score, the W-statistics are 1.0 for both ROC-Net and MARCO-Net with corresponding p -values of 0.011, which confirms that the harmonic balance between precision and recall in the proposed models is significantly superior to that of CapsNet. The similar trend is observed for MCC with both models achieving W-statistics of 1.0 and p -values of 0.011, highlighting consistent improvements in correlation-based measures of classification quality. Overall, the consistently low W-statistics and p -values across all metrics provide strong evidence that both ROC-Net and MARCO-Net significantly outperform the baseline CapsNet. These findings statistically validate the robustness and reliability of the proposed enhancements in IDS for IoT systems. Moreover, the parameter $p < 0.05$ has been observed consistently in all performance indicators, and it is assumed that the enhancements that the proposed models provide are not only statistically significant but also practically relevant in the real-life IoT intrusion detection environment. Such statistically validated improvements, in operation environments, provide more dependable attack detection, reduced false alarms, and higher confidence in decision-making when under uncertainty, which improves the overall security posture of the IoT systems.

3.9. Local interpretable model-agnostic explanations of ROC-Net and MARCO-Net in IoT

In IoT environments, IDS models are often treated as black-boxes. LIME addresses this by approximating the complex model locally with an interpretable surrogate model, such as a sparse linear model [39]. Given a trained classifier $f: \mathbb{R}^d \rightarrow \{0, 1\}$ and an input instance $x \in \mathbb{R}^d$, LIME perturbs x to generate a neighborhood dataset $\{x'_1, x'_2, \dots, x'_n\}$ with corresponding black-box predictions $f(x'_i)$. It then fits a local surrogate model $g \in G$ (where G is the class of simple models, e.g., linear models or decision trees) by minimizing the following objective function using Eq. (12):

$$\xi(x) = \arg \min_{g \in G} \mathcal{L}(f, g, \pi_x) + \Omega(g), \quad (12)$$

where $\mathcal{L}(f, g, \pi_x)$ measures the fidelity of g to f in the locality defined by the kernel π_x (which assigns higher weights to samples closer to x), and $\Omega(g)$ is a complexity penalty that enforces interpretability.

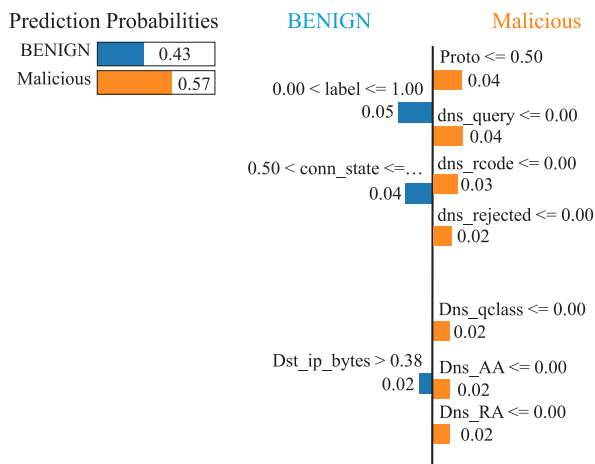


Fig. 11. LIME prediction for ROC-Net for intrusion detection in IoT.

In IoT IDS, this implies that for a specific traffic instance x (e.g., a packet with features like *protocol*, *duration*, *conn_state*), LIME explains the model's decision quantifying each feature's contributions in the linear approximation. For example, if $f(x) = 1$ (attack), LIME show that features such as high *dst_ip_bytes* or abnormal *dns_query* values contribute positively to the decision, thereby providing security experts with actionable insights. Thus, LIME bridges the gap between accuracy and interpretability in IoT, making black-box IDS more transparent and trustworthy.

3.9.1. LIME-based interpretability of ROC-Net predictions

Fig. 11 presents the LIME-based local interpretability of a classification decision for IDS in IoT systems. The prediction probabilities indicate that the instance has a probability of 0.43 of being classified as *benign* and 0.57 of being *malicious*. The local surrogate model explains these predictions using key IoT traffic features. The feature *label* with range $0.00 < label \leq 1.00$ contributes positively towards the benign class, while the feature *conn_state* with $0.50 < conn_state \leq \dots$ reinforces the benign prediction by reflecting a stable connection status in IoT communication. On the other hand, several features shift the prediction towards the malicious class. For example, when $proto \leq 0.50$, it suggests the use of an abnormal or nonstandard protocol, thereby increasing the likelihood of malicious activity. Similarly, conditions such as $dns_query \leq 0.00$, $dns_rcode \leq 0.00$, and $dns_rejected \leq 0.00$ indicate abnormal DNS behavior, including failed or rejected queries, which commonly signal malicious activities such as spoofing or DoS attacks in IoT environments. The feature $dst_ip_bytes > 0.38$ provides a benign contribution, as balanced outgoing traffic bytes typically represent normal device communication. Conversely, features like *dns_qclass*, *dns_AA*, and *dns_RA* with values ≤ 0.00 add weight to the malicious decision, indicating DNS configurations or responses inconsistent with benign traffic. Altogether, the plot demonstrates that although benign indicators such as stable connection states and outgoing traffic are present, the abnormal DNS patterns strongly influence the model to classify the instance as malicious. This insight is particularly critical in IoT IDS where DNS manipulation and protocol misuse are common attack vectors.

Fig. 12 presents the ROC-Net LIME summary plot, highlighting the most influential features behind the prediction for IDS in IoT traffic. The probability distribution shows that the instance is classified as malicious with higher confidence than benign. The feature *label* with range $0.00 < label \leq 1.00$ contributes positively towards benign classification, showing that the sample initially carries some characteristics of normal traffic. The feature *conn_state* with range $0.50 < conn_state \leq 0.83$ also supports the benign classification, as stable connection states in IoT

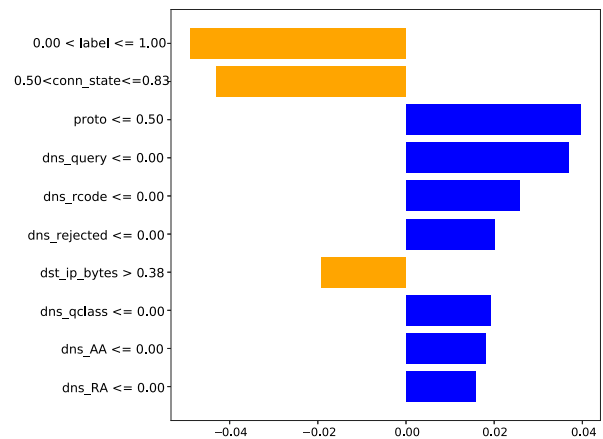


Fig. 12. LIME summary plot for ROC-Net for intrusion detection in IoT.

traffic typically represent normal device behavior. However, several features shift the decision towards malicious activity. For example, $proto \leq 0.50$ indicates use of a protocol that is potentially unusual or abnormal for the given context, raising suspicion of intrusion. Features such as $dns_query \leq 0.00$, $dns_rcode \leq 0.00$, and $dns_rejected \leq 0.00$ indicate abnormal DNS activity, including failed or rejected queries, which are often associated with spoofing, DoS, or reconnaissance attacks in IoT networks. The feature $dst_ip_bytes > 0.38$ provides a benign signal, since balanced outgoing traffic from IoT devices often corresponds to regular communication. Conversely, $dns_qclass \leq 0.00$, $dns_AA \leq 0.00$, and $dns_RA \leq 0.00$ add weight towards malicious classification, suggesting that abnormal or missing DNS response attributes are strongly linked with intrusion behavior. Overall, the summary plot demonstrates that while benign indicators such as connection stability and balanced traffic are present, abnormal DNS patterns exert a dominant influence, ultimately leading ROC-Net to classify the instance as malicious in the IoT IDS.

3.9.2. LIME-based interpretability of MARCO-Net predictions

The prediction probabilities for the MARCO-Net proposed model shown in Fig. 13 illustrate that the input instance has a probability of 0.66 of being classified as benign and 0.34 as malicious. This indicates higher confidence in the benign class, though malicious also receives a significant probability, suggesting the presence of mixed or overlapping patterns in the traffic. Among the influencing features, *conn_state* with a threshold of 0.08 plays the most significant role, as connection state is a fundamental indicator of whether IoT traffic follows normal session rules or exhibits abnormal termination and reset behaviors. The feature *proto* with a contribution of 0.50 further highlights the type of communication protocol used; abnormal or less frequently used protocols in IoT be associated with malicious intent. The feature *dns_query* with a weight of 0.02 captures the frequency of domain resolution requests, which in the IoT ecosystem often indicates attempts of devices to connect with external domains, sometimes linked to command and control servers in attacks. Similarly, *dns_RD* with negligible contribution represents the recursion desired flag, where malicious IoT devices often manipulate recursive queries for exfiltration. The features *dns_AA* and *dns_RA*, each contributing 0.01, reflect authoritative and recursion availability attributes that are usually benign in properly configured IoT networks but, if misused, often reveal malicious manipulation of DNS responses. Features like *dns_qtype* and *dns_qclass* with contributions close to zero indicate minimal role in this prediction, as the query type and class did not deviate significantly from normal IoT traffic. Traffic volume features like *dst_bytes* and *dst_ip_bytes* show no contribution, suggesting that packet sizes in this instance are consistent with benign IoT communication. Overall, the model leans towards classifying this

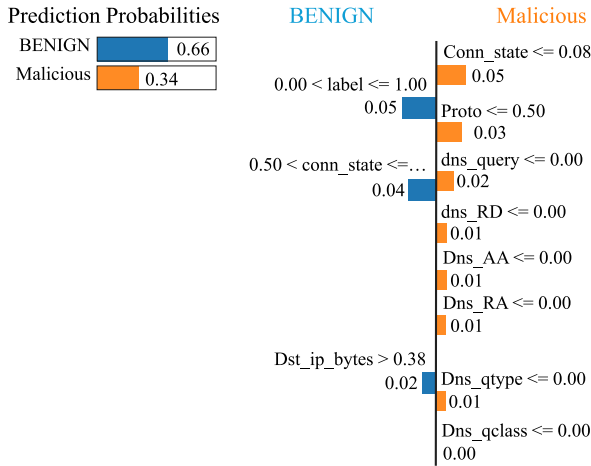


Fig. 13. LIME prediction for MARCO-Net for intrusion detection in IoT.

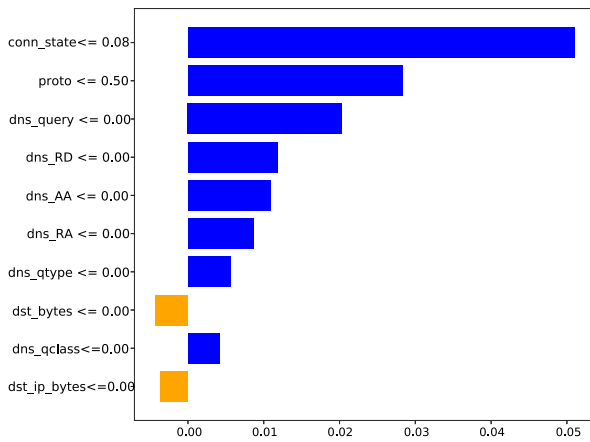


Fig. 14. LIME summary plot for MARCO-Net for intrusion detection in IoT.

traffic as benign due to the stability of most features, although the moderate influence of connection state and protocol indicates that the traffic exhibits some patterns similar to malicious IoT behavior. This demonstrates how MARCO-Net balances multiple low-level indicators to make nuanced distinctions in IoT security.

The LIME summary plot for MARCO-Net, presented in Fig. 14, illustrates the relative contributions of different features in shaping the model's decision for IDS in IoT traffic. Among the most influential features, *conn_state* with a threshold of 0.08 exerted the highest impact, signifying the importance of connection stability in IoT communication. Abnormal connection states often indicate reset or dropped sessions, which are common in malicious traffic. The feature *proto* with a contribution of 0.50 indicated the significance of communication protocol type, where non-standard or suspicious protocol usage can raise the likelihood of malicious classification. *dns_query* with a weight of 0.02 captured the role of DNS resolution requests, which in IoT often signals attempts to contact external domains, sometimes linked to botnet activity or command-and-control communication. The features *dns_RD*, *dns_AA*, and *dns_RA*, each contributing around 0.01, showed minimal but consistent effects; in normal IoT environments, these DNS flags remain stable, but their misuse or manipulation can reflect spoofing or cache poisoning attempts. Meanwhile, *dns_qtype* and *dns_qclass* had negligible contributions, suggesting that query type and class in this instance were aligned with normal traffic. Similarly, traffic volume features such as *dst_bytes* and *dst_ip_bytes* showed nearly zero impact, implying packet size and data exchange volumes did not deviate from benign IoT communication patterns.

Overall, the LIME summary plot confirms that MARCO-Net bases its classification primarily on connection state and protocol type while using DNS features to refine the decision boundary. Although benign signals are present, the influence of protocol and connection anomalies shows that the model remains cautious of potential threats, balancing detection sensitivity with reliable benign classification.

3.10. Shapley additive explanations of ROC-Net and MARCO-Net in IoT

SHAP provide explainability in IoT IDS by quantifying the contribution of each feature to the model's prediction [40,41]. Based on game theory, the SHAP value for a feature i is expressed using Eq. (13):

$$\phi_i = \sum_{S \subseteq N \setminus \{i\}} \frac{|S|!(|N| - |S| - 1)!}{|N|!} (f(S \cup \{i\}) - f(S)), \quad (13)$$

this equation computes the SHAP value ϕ_i by averaging the contribution of feature i over all possible subsets S of the feature set N that do not include i . The term $f(S \cup \{i\}) - f(S)$ measures how much adding feature i changes the model's prediction, while the factorial weighting ensures a fair contribution based on game-theoretic principles. The model output can be decomposed using Eq. (14):

$$f(x) = \phi_0 + \sum_{i=1}^M \phi_i, \quad (14)$$

where ϕ_0 is the baseline value and ϕ_i denotes the contribution of feature i . In IoT IDS, features such as *protocol type*, *connection state*, and *DNS queries* receive high positive SHAP values when pushing the decision towards malicious classification, while stable traffic indicators like balanced byte counts often receive negative values, favoring benign classification. This ensures that predictions in IoT environments are both accurate and transparent.

3.10.1. Interpretation of summary plots of proposed models

The SHAP summary plot for ROC-Net in Fig. 15 highlights the most influential features in IoT IDS, including *proto*, *duration*, *service*, *dst_bytes*, and *src_bytes*. The feature *proto* significantly impacts the classification, as unusual or suspicious protocol usage often indicates malicious activity in IoT communication. Abnormal connection *duration* reflects either prolonged malicious sessions or abrupt terminations inconsistent with normal IoT traffic. The *service* feature captures the type of network service requested, where irregular service usage often expose attempts of reconnaissance or exploitation. The feature *dst_bytes* shows how the volume of data sent to destination devices influences predictions, with excessively high or abnormally low values pointing towards data exfiltration or DoS behavior. Variations in *src_bytes* capture irregular outgoing traffic from IoT devices, which often corresponds to compromised nodes generating suspicious outgoing flows. SHAP shows that ROC-Net decisions rely on protocol misuse, abnormal traffic volume, and irregular connection behavior collectively signal potential intrusions in IoT networks.

The SHAP summary plot for MARCO-Net in Fig. 16 identifies the top contributing features as *proto*, *duration*, *src_ip.bytes*, *dns.qclass*, and *service*. The feature *proto* exerts strong influence, as abnormal or less common protocols often signify unauthorized access or suspicious IoT traffic. The feature *duration* highlights connection stability, where unusually long or short sessions often correspond to DoS or probing activities. The contribution of *src_ip.bytes* reflects irregular outgoing traffic from IoT devices, which can indicate compromised nodes participating in data exfiltration or botnet communication. Similarly, *dns.qclass* captures anomalies in DNS query classes, where unexpected values often reveal malicious manipulation of name resolution in IoT networks. Finally, *service* plays a critical role, since requested services often distinguish benign IoT activity from targeted attacks exploiting specific services. Overall, these features explain how MARCO-Net leverages protocol behavior, traffic patterns, and DNS irregularities to effectively identify intrusions in IoT environments.

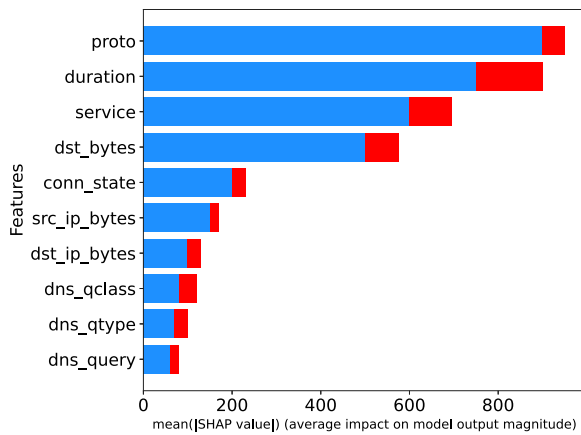


Fig. 15. SHAP summary plot for ROC-Net for intrusion detection in IoT.

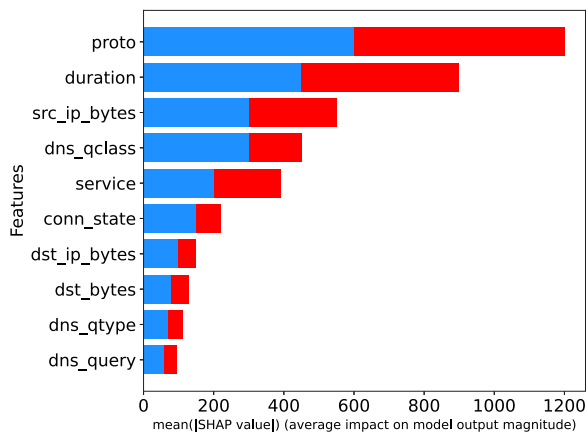


Fig. 16. SHAP summary plot for MARCO-Net for intrusion detection in IoT.

3.10.2. Interpretation of waterfall plots of proposed models

The SHAP waterfall plot for ROC-Net in Fig. 17 illustrates how individual features drive the prediction towards a malicious or benign class. The feature *src_ip_bytes* with a value of 0.22 contributes notably, since lower outgoing traffic from IoT devices often indicate abnormal or suppressed communication, often linked with compromised nodes. The feature *dns_qtype* with a value of 0 provides minimal contribution, since query types are consistent with normal IoT traffic behavior in this case. In contrast, the feature *proto* with a value of 0.45 plays a strong role, as non-standard or suspicious protocol usage frequently signals malicious activity in IoT networks. Similarly, the feature *service* with a value of 0.5 exerts significant influence, since irregular or unexpected service requests often reflect intrusion attempts such as reconnaissance or exploitation. Altogether, these features shape the ROC-Net prediction by showing how abnormal protocol and service patterns, along with irregular traffic volume, collectively increase the likelihood of classifying the IoT communication as malicious.

Fig. 18 shows the SHAP waterfall plot for MARCO-Net, highlighting the feature-wise contributions influencing the final prediction. The feature *dns_qtype* with a value of 0.05 shows a minor contribution, as normal query types typically align with benign IoT communication. In contrast, *proto* with a value of 0.62 exerts a dominant effect, since the use of unusual or suspicious protocols strongly increases the likelihood of malicious classification in IoT traffic. The feature *dst_bytes* with a value of 0.12 also contribute, as abnormal volumes of data sent to destination nodes often represent DoS or exfiltration attempts. Similarly, *service* with a value of 0.31 plays a significant role, as irregular service requests often indicate probing or targeted exploitation in IoT systems.

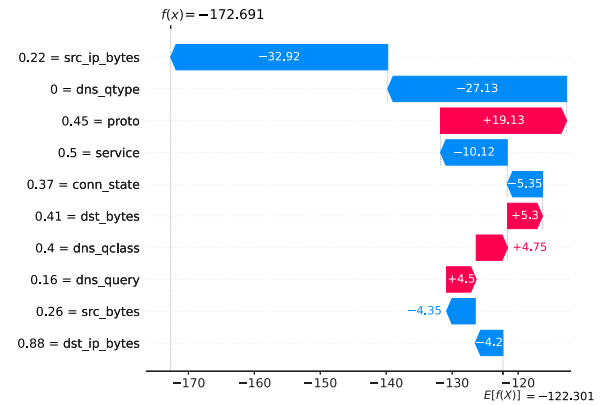


Fig. 17. SHAP waterfall plot for ROC-Net for intrusion detection in IoT.

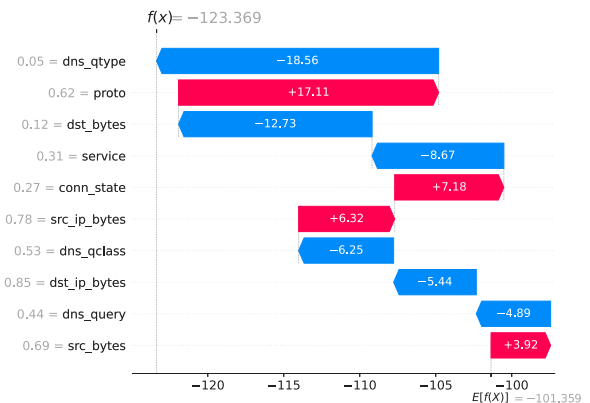


Fig. 18. SHAP waterfall plot for MARCO-Net for intrusion detection in IoT.

Together, these features show how MARCO-Net balances DNS query analysis, protocol behavior, traffic volume, and service type to deliver an interpretable and reliable IDS outcome in IoT environments.

3.10.3. Interpretation of force plots of proposed models

The Fig. 19 illustrates the contribution of different IoT network traffic features toward the prediction outcome in the model. The feature *duration* describes the length of a flow and exceptional long or short values of the duration usually signify malicious actions like a DoS attack or sudden loss of connection. The *proto* feature identifies the application or transport protocol used, and this is vital since certain attacks are more likely to attack certain protocols. The frequency and kind of domain name lookups are also noted in the *dns_RA* feature but other features are *dns_qclass* and *dns_AA* which can give additional resolution on query class and authoritative answer flags which may indicate some form of crafted DNS behavior in either tunneling or data exfiltration. The feature *dns_RA* indicates recursion availability, which often be exploited in amplification attacks. Similarly, *src_bytes* and *src_ip_bytes* measure the data volume sent by the source, and abnormal values can point to flooding or exfiltration attempts. The *service* feature is the kind of application service requested, which is vital in identifying whether traffic conforms to the expected activities of legitimate IoT devices or it has gone rogue towards malicious communication patterns.

The Fig. 20 demonstrates the impact of individual IoT traffic features on the model prediction, where the horizontal axis represents the contribution function $f(x)$ and the relative positioning indicates whether a feature increases or decreases the likelihood of a certain outcome. The attribute *duration* is once again the duration of a flow of communication, which could be significant in setting aside a normal exchange between devices and a long or abnormally short malicious



Fig. 19. SHAP force plot for ROC-Net for intrusion detection in IoT.

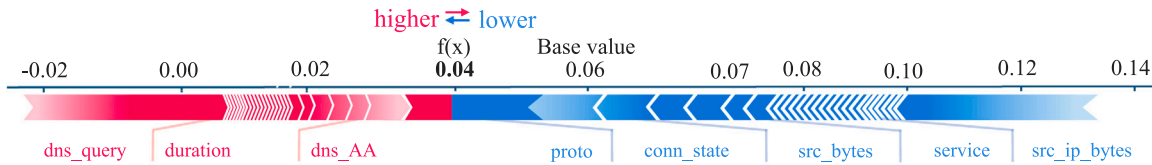


Fig. 20. SHAP force plot for MARCO-Net for intrusion detection in IoT.

session. The protocol used is defined by the *proto* feature and it is an important choice because some attacks on the IoT may use vulnerabilities in protocol processing. The *dns_query* variable captures the request patterns made to domain name servers, while *dns_qclass* and *dns_AA* reflect query classes and authoritative responses that are often manipulated in DNS-based attacks. The feature *dns_RA* shows recursion availability, which is a factor in detecting possible amplification misuse. The attribute *src_bytes* measures the volume of outgoing traffic from the IoT node, and extreme values often signal data exfiltration or flooding. The *conn_state* feature reflects the overall state of the connection, revealing suspicious terminations or resets. The feature *service* indicates the kind of network service being interacted with, and the attribute *src_ip_bytes* indicates the amount of volume of bytes directly associated with the source IP, both of which are important in distinguishing between legitimate and hostile IoT device usage. In general, the contribution of features in this plot highlight how traffic duration, protocol, DNS behavior, and byte volumes together affect the decision of the model to detect in the IoT context.

3.11. Comparative analysis of LIME and SHAP

The interpretability of the ML models is critical in providing trust, transparency, and reliability in the context of IoT applications when there is a large amount of heterogeneous data gathered by interconnected devices. LIME offers local interpretability, which means that the model is approximated by a small number of surrogate models to estimate its behavior in the vicinity of a single instance. This renders it efficient in applications like anomaly detection in IoT networks, where quick clarifications on individual forecasts are essential. But, LIME is based on the perturbation method, it does not provide consistent explanations between runs, making it less confident in the highly sensitive IoT systems.

Conversely, SHAP is based on game theory and provides local and global explainability through the assignment of the importance of features in all predictions. In IoT-based systems, which include predictive maintenance, IDS, and health monitoring, SHAP provides a stronger insight of the contribution of each sensor characteristic to model results. Although computationally more expensive than LIME, SHAP ensures stability and fairness in explanations, which is crucial in IoT environments where decisions often directly impact system performance and user safety. Overall, while LIME is lightweight and useful for quick insights, SHAP provides a more reliable and comprehensive interpretability framework in IoT systems.

4. Conclusion

In this study, we presented an efficient and intelligent DOAL-IDS framework tailored for IoT environments, addressing the core challenges faced by traditional methods. The integration of VT for feature selection and ProWRAS for data balancing ensured cleaner and

more representative training data. CapsNet, optimized through the RSA named as ROC-Net, provided superior classification capability, which was further enhanced by MBAL in the form of MARCO-Net. The proposed models, ROC-Net and MARCO-Net, outperform traditional models by achieving an accuracy of 0.87 and 0.90, F1-score of 0.87 and 0.90, MCC and Cohen's Kappa of 0.75 and 0.81, log loss of 0.70 and 0.52, and Hamming loss of 0.12 and 0.09, respectively. These results highlight the potential of combining optimization algorithms and active learning with deep models to build efficient IDS for real-world IoT deployments. To further make these findings more reliable, they were extensively verified against 10-FCV, and the same data showed the same performance in different data partitions. Moreover, the Wilcoxon Signed-Rank Test was also used to provide a statistical validation of the importance of the observed improvements in comparison with the baseline models. Beyond accuracy and robustness, this study also emphasizes interpretability: SHAP and LIME were employed to provide meaningful explanations of the models' predictions.

4.1. Limitations

Despite the promising predictive performance and interpretability of the proposed framework, it is important to mention a number of limitations. First, the study is tested on one dataset and this limits its applicability to other datasets and real-life multi-center clinical settings. Second, explainability tools like LIME and SHAP are combined to achieve transparency, which is not yet proven to be reliable in the domain, which may be restrictive to clinical use. Third, the framework has not been tested on sensitive clinical data and therefore its use is limited in clinical practice in real-time. Lastly, even though optimization and active learning can decrease overfitting, the model frequently still struggles with scalability and robustness in case of larger and more heterogeneous data.

4.2. Future work

Further development can be directed to overcome the limitations described, as well as expand the scope of the given work. To begin with, it would be more robust and generalizable to assess the framework on various datasets in different sources and real-life clinical setting. Second, the clinical credibility of the model can be improved by using domain experts to confirm the explanations produced by LIME and SHAP. Third, it would be beneficial to apply the framework to real-time streams of clinical data and electronic health records to make it more practical. Also, exploring lightweight architectures and distributed strategies of learning would enhance scalability, whereas considering the more advanced interpretability methods would contribute to transparency and trust that users have in making important healthcare decisions.

CRedit authorship contribution statement

Aymin Javed: Writing – original draft, Visualization, Validation, Methodology. **Nadeem Javaid:** Writing – review & editing, Supervision, Methodology, Conceptualization. **Imran Ahmed:** Validation, Formal analysis, Datacuration, Conceptualization. **Badr Alsamani:** Funding acquisition, Validation, Investigation, Formal analysis, Conceptualization. **Abdul Khader Jilani Saudagar:** Resources, Formal analysis, Data curation.

Funding statement

This work was supported and funded by the Deanship of Scientific Research at Imam Mohammad Ibn Saud Islamic University (IMSIU) (grant number IMSIU-DDRSP2604).

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

The authors extend their appreciation to the Deanship of Scientific Research at Imam Mohammad Ibn Saud Islamic University (IMSIU) for funding this work through (grant number IMSIU-DDRSP2604).

Data availability

The dataset used in this study is publicly accessible and can be found at: <https://www.kaggle.com/datasets/fadiabuzwayed/ton-iot-train-test-network>.

References

- [1] S. Soliman, W. Oudah, A. Aljuhani, Deep learning-based intrusion detection approach for securing industrial Internet of Things, *Alex. Eng. J.* 81 (2023) 371–383.
- [2] A. Heidari, M.A. Jabrael Jamali, Internet of Things intrusion detection systems: a comprehensive review and future directions, *Clust. Comput.* 26 (6) (2023) 3753–3780.
- [3] M. Bhavsar, K. Roy, J. Kelly, O. Olusola, Anomaly-based intrusion detection system for IoT application, *Discov. Internet Things* 3 (1) (2023) 5.
- [4] F. Qiu, A. Kumar, J. Hu, P. Sharma, Y.B. Tang, Y. Xu Xiang, J. Hong, A review on integrating IoT, IIoT, and industry 4.0: A pathway to smart manufacturing and digital transformation, *IET Inf. Secur.* 2025 (1) (2025) 9275962.
- [5] A. Verma, V. Ranga, Machine learning based intrusion detection systems for IoT applications, *Wirel. Pers. Commun.* 111 (4) (2020) 2287–2310.
- [6] V. Merlino, D. Allegra, Energy-based approach for attack detection in IoT devices: A survey, *Internet Things* 27 (2024) 101306.
- [7] M. Ammar, N. Javaid, A.K.J. Saudagar, I. Ahmed, An optimized deep and active learning oriented framework for intrusion detection in internet of sensor things, *Ain Shams Eng. J.* 16 (10) (2025) 103607.
- [8] E. Benkhelifa, T. Welsh, W. Hamouda, A critical review of practices and challenges in intrusion detection systems for IoT: toward universal and resilient systems, *IEEE Commun. Surv. Tutor.* 20 (4) (2018) 3496–3509.
- [9] Y. Chen, Q. Lin, W. Wei, J. Ji, K.C. Wong, C.A.C. Coello, Intrusion detection using multi-objective evolutionary convolutional neural network for internet of things in fog computing, *Knowl.-Based Syst.* 244, 108505.
- [10] L. Xiaojia, Z. Guosheng, W. Yang, N. Ke, Improved internet of things intrusion detection model for CNN and RNN, *J. Comput. Eng. Appl.* 59 (14) (2023).
- [11] H.H. Pajouh, R. Javidan, R. Khayami, A. Dehghantanha, K.K.R. Choo, A two-layer dimension reduction and two-tier classification model for anomaly-based intrusion detection in IoT backbone networks, *IEEE Trans. Emerg. Top. Comput.* 7 (2) (2016) 314–323.
- [12] B. Sharma, L. Sharma, C. Lal, S. Roy, Anomaly based network intrusion detection for IoT attacks using deep learning technique, *Comput. Electr. Eng.* 107 (2023) 108626.
- [13] Y. Chen, M. Mir, An intrusion detection system for industrial IoT using CNN-LSTM and reptile search algorithm, *Clust. Comput.* 28 (7) (2025) 452.
- [14] M. Ge, X. Fu, N. Syed, Z. Baig, G. Teo, A. Robles-Kelly, Deep learning-based intrusion detection for IoT networks, in: 2019 IEEE 24th Pacific Rim International Symposium on Dependable Computing, PRDC, IEEE, 2019, pp. 256–25609.
- [15] L.N. CheSuh, R.A. Fernandez-Diaz, J.M. Alija-Perez, C. Benavides-Cuellar, H. Alaiz-Moreton, Improve quality of service for the Internet of Things using blockchain & machine learning algorithms, *Internet Things* 26 (2024) 101123.
- [16] K. Alemerien, S. Al-Suhemat, M. Almahadin, Towards optimized machine-learning-driven intrusion detection for Internet of Things applications, *Int. J. Inf. Technol.* 16 (8) (2024) 4981–4994.
- [17] N. Soltani, A.M. Rahmani, M. Bohloul, M. Hosseinzadeh, Robust intrusion detection for network communication on the Internet of Things: a hybrid machine learning approach, *Clust. Comput.* 27 (7) (2024) 9975–9991.
- [18] H. Zhang, Development of an intelligent intrusion detection system for IoT networks using deep learning, *Discov. Internet Things* 5 (1) (2025) 74.
- [19] M.A. Talukder, M. Khalid, N. Sultana, A hybrid machine learning model for intrusion detection in wireless sensor networks leveraging data balancing and dimensionality reduction, *Sci. Rep.* 15 (1) (2025) 4617.
- [20] C.R. Borra, R.V. Rayala, Z. Khan, S. Cheekati, Enhancing IoT network security: Machine learning-based intrusion detection using vortex search optimization and SMOTE for improved classification, in: 2025 International Conference on Intelligent and Cloud Computing, ICoCC, IEEE, 2025, pp. 1–7.
- [21] A. Javed, N. Javaid, Z. Ali, N. Nasser, OASIS: Optimized active sampling for intrusion detection in IoT systems, in: 2025 12th International Conference on Wireless Networks and Mobile Communications, WINCOM, IEEE, 2025, pp. 1–6.
- [22] A. Alsaedi, N. Moustafa, Z. Tari, A. Mahmood, A. Anwar, TON_IoT telemetry dataset: A new generation dataset of IoT and IIoT for data-driven intrusion detection systems, *IEEE Access* 8 (2020) 165130–165150.
- [23] M.O. Kaya, M. Ozdem, R. Das, A new hybrid approach combining GCN and LSTM for real-time anomaly detection from dynamic computer network data, *Comput. Netw.* (2025) 111372.
- [24] W. Song, X. Zhu, S. Ren, W. Tan, Y. Peng, A hybrid blockchain and machine learning approach for intrusion detection system in industrial Internet of Things, *Alex. Eng. J.* 127 (2025) 619–627.
- [25] M.A.F.A. Fida, T. Ahmad, M. Ntahobari, Variance threshold as early screening to boruta feature selection for intrusion detection system, in: 2021 13th International Conference on Information & Communication Technology and System, ICTS, IEEE, 2021, pp. 46–50.
- [26] S. Bej, K. Schulz, P. Srivastava, M. Wolfien, O. Wolkenhauer, A multi-schematic classifier-independent oversampling approach for imbalanced datasets, *IEEE Access* 9 (2021) 123358–123374.
- [27] L. Abualigah, M. Abd Elaziz, P. Sumari, Z.W. Geem, A.H. Gandomi, Reptile search algorithm (RSA): A nature-inspired meta-heuristic optimizer, *Expert Syst. Appl.* 191 (2022) 116158.
- [28] B. Sasmal, A.G. Hussien, A. Das, K.G. Dhal, R. Saha, Reptile search algorithm: Theory, variants, applications, and performance evaluation, *Arch. Comput. Methods Eng.* 31 (1) (2024) 521–549.
- [29] B. Zhang, L. Li, S. Wang, S. Cai, Z.J. Zha, Q. Tian, Q. Huang, Inductive state-relabeling adversarial active learning with heuristic clique rescaling, *IEEE Trans. Pattern Anal. Mach. Intell.* (2024).
- [30] H.R. Sayegh, W. Dong, A.M. Al-madani, Enhanced intrusion detection with LSTM-based model, feature selection, and SMOTE for imbalanced data, *Appl. Sci.* 14 (2) (2024) 479.
- [31] G. Abdelmoumin, D.B. Rawat, A. Rahman, Studying imbalanced learning for anomaly-based intelligent IDS for mission-critical Internet of Things, *J. Cybersecur. Priv.* 3 (4) (2023) 706–743.
- [32] U. Ahmed, J.C.W. Lin, G. Srivastava, A resource allocation deep active learning based on load balancer for network intrusion detection in SDN sensors, *Comput. Commun.* 184 (2022) 56–63.
- [33] A. Guerra-Manzanares, H. Bahsi, On the application of active learning for efficient and effective IoT botnet detection, *Future Gener. Comput. Syst.* 141 (2023) 40–53.
- [34] J. Nayak, S.K. Meher, A. Souri, B. Naik, S. Vimal, Extreme learning machine and bayesian optimization-driven intelligent framework for IoMT cyber-attack detection, *J. Supercomput.* 78 (13) (2022) 14866–14891.
- [35] S. Perumalla, S. Chatterjee, A.S. Kumar, Modelling of oppositional aquila optimizer with machine learning enabled secure access control in internet of drones environment, *Theoret. Comput. Sci.* 941 (2023) 39–54.
- [36] M. Ammar, N. Javaid, A. Arishi, An AI explained systematic modular approach for enhanced electricity theft detection for urbanized smart grid environment, *Alex. Eng. J.* 129 (2025) 998–1023.
- [37] I.K. Nti, O. Nyarko-Boateng, J. Aning, Performance of machine learning algorithms with different k values in K-fold cross-validation, *Int. J. Inf. Technol. Comput. Sci.* 13 (6) (2021) 61–71.
- [38] S.M. Taheri, G. Hesamian, A generalization of the wilcoxon signed-rank test and its applications, *Statist. Papers* 54 (2) (2013) 457–470.
- [39] Y.H. Hung, C.Y. Lee, BMB-LIME: LIME with modeling local nonlinearity and uncertainty in explainability, *Knowl.-Based Syst.* 294 (2024) 111732.

- [40] T.T.H. Le, H. Kim, H. Kang, H. Kim, Classification and explanation for intrusion detection system based on ensemble trees and SHAP method, *Sensors* 22 (3) (2022) 1154.
- [41] Z.A. Khan, N. Javaid, A. Saeed, I. Ahmed, F.A. Khan, Towards IoT device privacy & data integrity through decentralized storage with blockchain and predicting malicious entities by stacked machine learning, *Internet Things* (2025) 101642.